

C H A P T E R

15

OBJECT-ORIENTED
PROGRAMMING

CONTENTS

Section 15.1	OOP: An Overview	592
Section 15.2	Defining Base and Derived Classes	594
Section 15.3	Virtual Functions	603
Section 15.4	Abstract Base Classes	608
Section 15.5	Access Control and Inheritance	611
Section 15.6	Class Scope under Inheritance	617
Section 15.7	Constructors and Copy Control	622
Section 15.8	Containers and Inheritance	630
Section 15.9	Text Queries Revisited	634
Chapter Summary		649
Defined Terms		649

Object-oriented programming is based on three fundamental concepts: data abstraction, which we covered in Chapter 7, and inheritance and dynamic binding, which we’ll cover in this chapter.

Inheritance and dynamic binding affect how we write our programs in two ways: They make it easier to define new classes that are similar, but not identical, to other classes, and they make it easier for us to write programs that can ignore the details of how those similar types differ.

Many applications include concepts that are related to but slightly different from one another. For example, our bookstore might offer different pricing strategies for different books. Some books might be sold only at a given price. Others might be sold subject to a discount. We might give a discount to purchasers who buy a specified number of copies of the book. Or we might give a discount for only the first few copies purchased but charge full price for any bought beyond a given limit, and so on. Object-oriented programming (OOP) is a good match to this kind of application.



15.1 OOP: An Overview

The key ideas in **object-oriented programming** are data abstraction, inheritance, and dynamic binding. Using data abstraction, we can define classes that separate interface from implementation (Chapter 7). Through inheritance, we can define classes that model the relationships among similar types. Through dynamic binding, we can use objects of these types while ignoring the details of how they differ.

Inheritance

Classes related by **inheritance** form a hierarchy. Typically there is a **base class** at the root of the hierarchy, from which the other classes inherit, directly or indirectly. These inheriting classes are known as **derived classes**. The base class defines those members that are common to the types in the hierarchy. Each derived class defines those members that are specific to the derived class itself.

To model our different kinds of pricing strategies, we'll define a class named `Quote`, which will be the base class of our hierarchy. A `Quote` object will represent undiscounted books. From `Quote` we will inherit a second class, named `Bulk_quote`, to represent books that can be sold with a quantity discount.

These classes will have the following two member functions:

- `isbn()`, which will return the ISBN. This operation does not depend on the specifics of the inherited class(es); it will be defined only in class `Quote`.
- `net_price(size_t)`, which will return the price for purchasing a specified number of copies of a book. This operation is type specific; both `Quote` and `Bulk_quote` will define their own version of this function.

In C++, a base class distinguishes functions that are type dependent from those that it expects its derived classes to inherit without change. The base class defines as **virtual** those functions it expects its derived classes to define for themselves. Using this knowledge, we can start to write our `Quote` class:

```
class Quote {
public:
    std::string isbn() const;
    virtual double net_price(std::size_t n) const;
};
```

A derived class must specify the class(es) from which it intends to inherit. It does so in a **class derivation list**, which is a colon followed by a comma-separated list of base classes each of which may have an optional access specifier:

```
class Bulk_quote : public Quote { // Bulk_quote inherits from Quote
public:
    double net_price(std::size_t) const override;
};
```

Because `Bulk_quote` uses `public` in its derivation list, we can use objects of type `Bulk_quote` *as if* they were `Quote` objects.

A derived class must include in its own class body a declaration of all the virtual functions it intends to define for itself. A derived class may include the `virtual` keyword on these functions but is not required to do so. For reasons we'll explain in § 15.3 (p. 606), the new standard lets a derived class explicitly note that it intends a member function to **override** a virtual that it inherits. It does so by specifying `override` after its parameter list.

Dynamic Binding

Through **dynamic binding**, we can use the same code to process objects of either type `Quote` or `Bulk_quote` interchangeably. For example, the following function prints the total price for purchasing the given number of copies of a given book:

```
// calculate and print the price for the given number of copies, applying any discounts
double print_total(ostream &os,
                  const Quote &item, size_t n)
{
    // depending on the type of the object bound to the item parameter
    // calls either Quote::net_price or Bulk_quote::net_price
    double ret = item.net_price(n);
    os << "ISBN: " << item.isbn() // calls Quote::isbn
        << " # sold: " << n << " total due: " << ret << endl;
    return ret;
}
```

This function is pretty simple—it prints the results of calling `isbn` and `net_price` on its parameter and returns the value calculated by the call to `net_price`.

Nevertheless, there are two interesting things about this function: For reasons we'll explain in § 15.2.3 (p. 601), because the `item` parameter is a reference to `Quote`, we can call this function on either a `Quote` object or a `Bulk_quote` object. And, for reasons we'll explain in § 15.2.1 (p. 594), because `net_price` is a virtual function, and because `print_total` calls `net_price` through a reference, the version of `net_price` that is run will depend on the type of the object that we pass to `print_total`:

```
// basic has type Quote; bulk has type Bulk_quote
print_total(cout, basic, 20); // calls Quote version of net_price
print_total(cout, bulk, 20); // calls Bulk_quote version of net_price
```

The first call passes a `Quote` object to `print_total`. When `print_total` calls `net_price`, the `Quote` version will be run. In the next call, the argument is a

Bulk_quote, so the Bulk_quote version of net_price (which applies a discount) will be run. Because the decision as to which version to run depends on the type of the argument, that decision can't be made until run time. Therefore, dynamic binding is sometimes known as **run-time binding**.



In C++, dynamic binding happens when a virtual function is called through a reference (or a pointer) to a base class.

15.2 Defining Base and Derived Classes

In many, but not all, ways base and derived classes are defined like other classes we have already seen. In this section, we'll cover the basic features used to define classes related by inheritance.



15.2.1 Defining a Base Class

We'll start by completing the definition of our Quote class:

```
class Quote {
public:
    Quote() = default; // = default see § 7.1.4 (p. 264)
    Quote(const std::string &book, double sales_price):
        bookNo(book), price(sales_price) { }
    std::string isbn() const { return bookNo; }
    // returns the total sales price for the specified number of items
    // derived classes will override and apply different discount algorithms
    virtual double net_price(std::size_t n) const
        { return n * price; }
    virtual ~Quote() = default; // dynamic binding for the destructor
private:
    std::string bookNo; // ISBN number of this item
protected:
    double price = 0.0; // normal, undiscounted price
};
```

The new parts in this class are the use of virtual on the net_price function and the destructor, and the protected access specifier. We'll explain virtual destructors in § 15.7.1 (p. 622), but for now it is worth noting that classes used as the root of an inheritance hierarchy almost always define a virtual destructor.



Base classes ordinarily should define a virtual destructor. Virtual destructors are needed even if they do no work.

Member Functions and Inheritance

Derived classes inherit the members of their base class. However, a derived class needs to be able to provide its own definition for operations, such as net_price,

that are type dependent. In such cases, the derived class needs to **override** the definition it inherits from the base class, by providing its own definition.

In C++, a base class must distinguish the functions it expects its derived classes to override from those that it expects its derived classes to inherit without change. The base class defines as **virtual** those functions it expects its derived classes to override. When we call a virtual function *through a pointer or reference*, the call will be dynamically bound. Depending on the type of the object to which the reference or pointer is bound, the version in the base class or in one of its derived classes will be executed.

A base class specifies that a member function should be dynamically bound by preceding its declaration with the keyword `virtual`. Any nonstatic member function (§ 7.6, p. 300), other than a constructor, may be virtual. The `virtual` keyword appears only on the declaration inside the class and may not be used on a function definition that appears outside the class body. A function that is declared as `virtual` in the base class is implicitly `virtual` in the derived classes as well. We'll have more to say about virtual functions in § 15.3 (p. 603).

Member functions that are not declared as `virtual` are resolved at compile time, not run time. For the `isbn` member, this is exactly the behavior we want. The `isbn` function does not depend on the details of a derived type. It behaves identically when run on a `Quote` or `Bulk_quote` object. There will be only one version of the `isbn` function in our inheritance hierarchy. Thus, there is no question as to which function to run when we call `isbn()`.

Access Control and Inheritance

A derived class inherits the members defined in its base class. However, the member functions in a derived class may not necessarily access the members that are inherited from the base class. Like any other code that uses the base class, a derived class may access the `public` members of its base class but may not access the `private` members. However, sometimes a base class has members that it wants to let its derived classes use while still prohibiting access to those same members by other users. We specify such members after a **protected** access specifier.

Our `Quote` class expects its derived classes to define their own `net_price` function. To do so, those classes need access to the `price` member. As a result, `Quote` defines that member as `protected`. Derived classes are expected to access `bookNo` in the same way as ordinary users—by calling the `isbn` function. Hence, the `bookNo` member is `private` and is inaccessible to classes that inherit from `Quote`. We'll have more to say about `protected` members in § 15.5 (p. 611).

EXERCISES SECTION 15.2.1

Exercise 15.1: What is a virtual member?

Exercise 15.2: How does the `protected` access specifier differ from `private`?

Exercise 15.3: Define your own versions of the `Quote` class and the `print_total` function.



15.2.2 Defining a Derived Class

A derived class must specify from which class(es) it inherits. It does so in its **class derivation list**, which is a colon followed by a comma-separated list of names of previously defined classes. Each base class name may be preceded by an optional access specifier, which is one of `public`, `protected`, or `private`.

A derived class must declare each inherited member function it intends to override. Therefore, our `Bulk_quote` class must include a `net_price` member:

```
class Bulk_quote : public Quote { // Bulk_quote inherits from Quote
    Bulk_quote() = default;
    Bulk_quote(const std::string&, double, std::size_t, double);
    // overrides the base version in order to implement the bulk purchase discount policy
    double net_price(std::size_t) const override;
private:
    std::size_t min_qty = 0; // minimum purchase for the discount to apply
    double discount = 0.0; // fractional discount to apply
};
```

Our `Bulk_quote` class inherits the `isbn` function and the `bookNo` and `price` data members of its `Quote` base class. It defines its own version of `net_price` and has two additional data members, `min_qty` and `discount`. These members specify the minimum quantity and the discount to apply once that number of copies are purchased.

We'll have more to say about the access specifier used in a derivation list in § 15.5 (p. 612). For now, what's useful to know is that the access specifier determines whether users of a derived class are allowed to know that the derived class inherits from its base class.

When the derivation is `public`, the `public` members of the base class become part of the interface of the derived class as well. In addition, we can bind an object of a publicly derived type to a pointer or reference to the base type. Because we used `public` in the derivation list, the interface to `Bulk_quote` implicitly contains the `isbn` function, and we may use a `Bulk_quote` object where a pointer or reference to `Quote` is expected.

Most classes inherit directly from only one base class. This form of inheritance, known as “single inheritance,” forms the topic of this chapter. § 18.3 (p. 802) will cover classes that have derivation lists with more than one base class.

Virtual Functions in the Derived Class

Derived classes frequently, but not always, override the virtual functions that they inherit. If a derived class does not override a virtual from its base, then, like any other member, the derived class inherits the version defined in its base class.

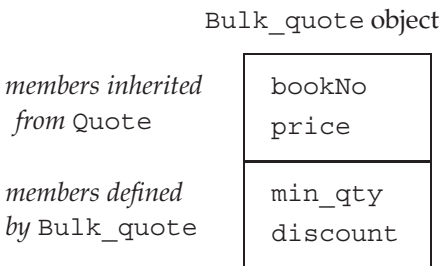
A derived class may include the `virtual` keyword on the functions it overrides, but it is not required to do so. For reasons we'll explain in § 15.3 (p. 606), the new standard lets a derived class explicitly note that it intends a member function to override a virtual that it inherits. It does so by specifying `override` after the parameter list, or after the `const` or reference qualifier(s) if the member is a `const` (§ 7.1.2, p. 258) or reference (§ 13.6.3, p. 546) function.

Derived-Class Objects and the Derived-to-Base Conversion

A derived object contains multiple parts: a subobject containing the (nonstatic) members defined in the derived class itself, plus subobjects corresponding to each base class from which the derived class inherits. Thus, a `Bulk_quote` object will contain four data elements: the `bookNo` and `price` data members that it inherits from `Quote`, and the `min_qty` and `discount` members, which are defined by `Bulk_quote`.

Although the standard does not specify how derived objects are laid out in memory, we can think of a `Bulk_quote` object as consisting of two parts as represented in Figure 15.1.

Figure 15.1: Conceptual Structure of a `Bulk_quote` Object



The base and derived parts of an object are not guaranteed to be stored contiguously.

Figure 15.1 is a conceptual, not physical, representation of how classes work.

Because a derived object contains subparts corresponding to its base class(es), we can use an object of a derived type *as if* it were an object of its base type(s). In particular, we can bind a base-class reference or pointer to the base-class part of a derived object.

```
Quote item;           // object of base type
Bulk_quote bulk;      // object of derived type
Quote *p = &item;     // p points to a Quote object
p = &bulk;            // p points to the Quote part of bulk
Quote &r = bulk;      // r bound to the Quote part of bulk
```

This conversion is often referred to as the **derived-to-base** conversion. As with any other conversion, the compiler will apply the derived-to-base conversion implicitly (§ 4.11, p. 159).

The fact that the derived-to-base conversion is implicit means that we can use an object of derived type or a reference to a derived type when a reference to the base type is required. Similarly, we can use a pointer to a derived type where a pointer to the base type is required.

The fact that a derived object contains subobjects for its base classes is key to how inheritance works.

Derived-Class Constructors

Although a derived object contains members that it inherits from its base, it cannot directly initialize those members. Like any other code that creates an object of the base-class type, a derived class must use a base-class constructor to initialize its base-class part.



Each class controls how its members are initialized.

The base-class part of an object is initialized, along with the data members of the derived class, during the initialization phase of the constructor (§ 7.5.1, p. 288). Analogously to how we initialize a member, a derived-class constructor uses its constructor initializer list to pass arguments to a base-class constructor. For example, the `Bulk_quote` constructor with four parameters:

```
Bulk_quote(const std::string& book, double p,
           std::size_t qty, double disc) :
    Quote(book, p), min_qty(qty), discount(disc) { }
    // as before
};
```

passes its first two parameters (representing the ISBN and price) to the `Quote` constructor. That `Quote` constructor initializes the `Bulk_quote`'s base-class part (i.e., the `bookNo` and `price` members). When the (empty) `Quote` constructor body completes, the base-class part of the object being constructed will have been initialized. Next the direct members, `min_qty` and `discount`, are initialized. Finally, the (empty) function body of the `Bulk_quote` constructor is run.

As with a data member, unless we say otherwise, the base part of a derived object is default initialized. To use a different base-class constructor, we provide a constructor initializer using the name of the base class, followed (as usual) by a parenthesized list of arguments. Those arguments are used to select which base-class constructor to use to initialize the base-class part of the derived object.



The base class is initialized first, and then the members of the derived class are initialized in the order in which they are declared in the class.

Using Members of the Base Class from the Derived Class

A derived class may access the public and protected members of its base class:

```
// if the specified number of items are purchased, use the discounted price
double Bulk_quote::net_price(size_t cnt) const
{
    if (cnt >= min_qty)
        return cnt * (1 - discount) * price;
    else
        return cnt * price;
}
```


This function generates a discounted price: If the given quantity is more than `min_qty`, we apply the discount (which was stored as a fraction) to the price.

We'll have more to say about scope in § 15.6 (p. 617), but for now it's worth knowing that the scope of a derived class is nested inside the scope of its base class. As a result, there is no distinction between how a member of the derived class uses members defined in its own class (e.g., `min_qty` and `discount`) and how it uses members defined in its base (e.g., `price`).

KEY CONCEPT: RESPECTING THE BASE-CLASS INTERFACE

It is essential to understand that each class defines its own interface. Interactions with an object of a class-type should use the interface of that class, even if that object is the base-class part of a derived object.

As a result, derived-class constructors may not directly initialize the members of its base class. The constructor body of a derived constructor can assign values to its public or protected base-class members. Although it *can* assign to those members, it generally *should not* do so. Like any other user of the base class, a derived class should respect the interface of its base class by using a constructor to initialize its inherited members.

Inheritance and static Members

If a base class defines a static member (§ 7.6, p. 300), there is only one such member defined for the entire hierarchy. Regardless of the number of classes derived from a base class, there exists a single instance of each static member.

```
class Base {
public:
    static void statmem();
};
class Derived : public Base {
    void f(const Derived&);
};
```

static members obey normal access control. If the member is private in the base class, then derived classes have no access to it. Assuming the member is accessible, we can use a static member through either the base or derived:

```
void Derived::f(const Derived &derived_obj)
{
    Base::statmem();      // ok: Base defines statmem
    Derived::statmem();   // ok: Derived inherits statmem
    // ok: derived objects can be used to access static from base
    derived_obj.statmem(); // accessed through a Derived object
    statmem();             // accessed through this object
}
```

Declarations of Derived Classes

A derived class is declared like any other class (§ 7.3.3, p. 278). The declaration contains the class name but does not include its derivation list:

```
class Bulk_quote : public Quote; // error: derivation list can't appear here
class Bulk_quote;                // ok: right way to declare a derived class
```

The purpose of a declaration is to make known that a name exists and what kind of entity it denotes, for example, a class, function, or variable. The derivation list, and all other details of the definition, must appear together in the class body.

Classes Used as a Base Class

A class must be defined, not just declared, before we can use it as a base class:

```
class Quote; // declared but not defined
// error: Quote must be defined
class Bulk_quote : public Quote { ... };
```

The reason for this restriction should be easy to see: Each derived class contains, and may use, the members it inherits from its base class. To use those members, the derived class must know what they are. One implication of this rule is that it is impossible to derive a class from itself.

A base class can itself be a derived class:

```
class Base { /* ... */ };
class D1: public Base { /* ... */ };
class D2: public D1 { /* ... */ };
```

In this hierarchy, Base is a **direct base** to D1 and an **indirect base** to D2. A direct base class is named in the derivation list. An indirect base is one that a derived class inherits through its direct base class.

Each class inherits all the members of its direct base class. The most derived class inherits the members of its direct base. The members in the direct base include those it inherits from its base class, and so on up the inheritance chain. Effectively, the most derived object contains a subobject for its direct base and for each of its indirect bases.

Preventing Inheritance

Sometimes we define a class that we don't want others to inherit from. Or we might define a class for which we don't want to think about whether it is appropriate as a base class. Under the new standard, we can prevent a class from being used as a base by following the class name with `final`:

**C++
11**

```
class NoDerived final { /* */ }; // NoDerived can't be a base class
class Base { /* */ };
// Last is final; we cannot inherit from Last
class Last final : Base { /* */ }; // Last can't be a base class
class Bad : NoDerived { /* */ }; // error: NoDerived is final
class Bad2 : Last { /* */ }; // error: Last is final
```

EXERCISES SECTION 15.2.2

Exercise 15.4: Which of the following declarations, if any, are incorrect? Explain why.

```
class Base { ... };
(a) class Derived : public Derived { ... };
(b) class Derived : private Base { ... };
(c) class Derived : public Base;
```

Exercise 15.5: Define your own version of the `Bulk_quote` class.

Exercise 15.6: Test your `print_total` function from the exercises in § 15.2.1 (p. 595) by passing both `Quote` and `Bulk_quote` objects to that function.

Exercise 15.7: Define a class that implements a limited discount strategy, which applies a discount to books purchased up to a given limit. If the number of copies exceeds that limit, the normal price applies to those purchased beyond the limit.

15.2.3 Conversions and Inheritance



Understanding conversions between base and derived classes is essential to understanding how object-oriented programming works in C++.

Ordinarily, we can bind a reference or a pointer only to an object that has the same type as the corresponding reference or pointer (§ 2.3.1, p. 51, and § 2.3.2, p. 52) or to a type that involves an acceptable `const` conversion (§ 4.11.2, p. 162). Classes related by inheritance are an important exception: We can bind a pointer or reference to a base-class type to an object of a type derived from that base class. For example, we can use a `Quote&` to refer to a `Bulk_quote` object, and we can assign the address of a `Bulk_quote` object to a `Quote*`.

The fact that we can bind a reference (or pointer) to a base-class type to a derived object has a crucially important implication: When we use a reference (or pointer) to a base-class type, we don't know the actual type of the object to which the pointer or reference is bound. That object can be an object of the base class or it can be an object of a derived class.



Like built-in pointers, the smart pointer classes (§ 12.1, p. 450) support the derived-to-base conversion—we can store a pointer to a derived object in a smart pointer to the base type.

Static Type and Dynamic Type



When we use types related by inheritance, we often need to distinguish between the **static type** of a variable or other expression and the **dynamic type** of the object that expression represents. The static type of an expression is always known at compile time—it is the type with which a variable is declared or that an expression yields. The dynamic type is the type of the object in memory that the variable or expression represents. The dynamic type may not be known until run time.

For example, when `print_total` calls `net_price` (§ 15.1, p. 593):

```
double ret = item.net_price(n);
```

we know that the static type of `item` is `Quote&`. The dynamic type depends on the type of the argument to which `item` is bound. That type cannot be known until a call is executed at run time. If we pass a `Bulk_quote` object to `print_total`, then the static type of `item` will differ from its dynamic type. As we've seen, the static type of `item` is `Quote&`, but in this case the dynamic type is `Bulk_quote`.

The dynamic type of an expression that is neither a reference nor a pointer is always the same as that expression's static type. For example, a variable of type `Quote` is always a `Quote` object; there is nothing we can do that will change the type of the object to which that variable corresponds.



It is crucial to understand that the static type of a pointer or reference to a base class may differ from its dynamic type.

There Is No Implicit Conversion from Base to Derived ...

The conversion from derived to base exists because every derived object contains a base-class part to which a pointer or reference of the base-class type can be bound. There is no similar guarantee for base-class objects. A base-class object can exist either as an independent object or as part of a derived object. A base object that is not part of a derived object has only the members defined by the base class; it doesn't have the members defined by the derived class.

Because a base object might or might not be part of a derived object, there is no automatic conversion from the base class to its derived class(s):

```
Quote base;
Bulk_quote* bulkP = &base; // error: can't convert base to derived
Bulk_quote& bulkRef = base; // error: can't convert base to derived
```

If these assignments were legal, we might attempt to use `bulkP` or `bulkRef` to use members that do not exist in `base`.

What is sometimes a bit surprising is that we cannot convert from base to derived even when a base pointer or reference is bound to a derived object:

```
Bulk_quote bulk;
Quote *itemP = &bulk; // ok: dynamic type is Bulk_quote
Bulk_quote *bulkP = itemP; // error: can't convert base to derived
```

The compiler has no way to know (at compile time) that a specific conversion will be safe at run time. The compiler looks only at the static types of the pointer or reference to determine whether a conversion is legal. If the base class has one or more virtual functions, we can use a `dynamic_cast` (which we'll cover in § 19.2.1 (p. 825)) to request a conversion that is checked at run time. Alternatively, in those cases when we *know* that the conversion from base to derived is safe, we can use a `static_cast` (§ 4.11.3, p. 162) to override the compiler.



...and No Conversion between Objects

The automatic derived-to-base conversion applies only for conversions to a reference or pointer type. There is no such conversion from a derived-class type to the base-class type. Nevertheless, it is often possible to convert an object of a derived class to its base-class type. However, such conversions may not behave as we might want.

Remember that when we initialize or assign an object of a class type, we are actually calling a function. When we initialize, we're calling a constructor (§ 13.1.1, p. 496, and § 13.6.2, p. 534); when we assign, we're calling an assignment operator (§ 13.1.2, p. 500, and § 13.6.2, p. 536). These members normally have a parameter that is a reference to the `const` version of the class type.

Because these members take references, the derived-to-base conversion lets us pass a derived object to a base-class copy/move operation. These operations are not virtual. When we pass a derived object to a base-class constructor, the constructor that is run is defined in the base class. That constructor knows *only* about the members of the base class itself. Similarly, if we assign a derived object to a base object, the assignment operator that is run is the one defined in the base class. That operator also knows *only* about the members of the base class itself.

For example, our bookstore classes use the synthesized versions of copy and assignment (§ 13.1.1, p. 497, and § 13.1.2, p. 500). We'll have more to say about copy control and inheritance in § 15.7.2 (p. 623), but for now what's useful to know is that the synthesized versions memberwise copy or assign the data members of the class the same way as for any other class:

```
Bulk_quote bulk;    // object of derived type
Quote item(bulk);   // uses the Quote::Quote(const Quote&) constructor
item = bulk;        // calls Quote::operator=(const Quote&)
```

When `item` is constructed, the `Quote` copy constructor is run. That constructor knows only about the `bookNo` and `price` members. It copies those members from the `Quote` part of `bulk` and *ignores* the members that are part of the `Bulk_quote` portion of `bulk`. Similarly for the assignment of `bulk` to `item`; only the `Quote` part of `bulk` is assigned to `item`.

Because the `Bulk_quote` part is ignored, we say that the `Bulk_quote` portion of `bulk` is **sliced down**.



WARNING

When we initialize or assign an object of a base type from an object of a derived type, only the base-class part of the derived object is copied, moved, or assigned. The derived part of the object is ignored.

15.3 Virtual Functions



As we've seen, in C++ dynamic binding happens when a virtual member function is called through a reference or a pointer to a base-class type (§ 15.1, p. 593). Because we don't know which version of a function is called until run time, virtual functions must *always* be defined. Ordinarily, if we do not use a function, we don't

EXERCISES SECTION 15.2.3

Exercise 15.8: Define static type and dynamic type.

Exercise 15.9: When is it possible for an expression's static type to differ from its dynamic type? Give three examples in which the static and dynamic type differ.

Exercise 15.10: Recalling the discussion from § 8.1 (p. 311), explain how the program on page 317 that passed an `ifstream` to the `Sales_data` `read` function works.

KEY CONCEPT: CONVERSIONS AMONG TYPES RELATED BY INHERITANCE

There are three things that are important to understand about conversions among classes related by inheritance:

- The conversion from derived to base applies only to pointer or reference types.
- There is no implicit conversion from the base-class type to the derived type.
- Like any member, the derived-to-base conversion may be inaccessible due to access controls. We'll cover accessibility in § 15.5 (p. 613).

Although the automatic conversion applies only to pointers and references, most classes in an inheritance hierarchy (implicitly or explicitly) define the copy-control members (Chapter 13). As a result, we can often copy, move, or assign an object of derived type to a base-type object. However, copying, moving, or assigning a derived-type object to a base-type object copies, moves, or assigns *only* the members in the base-class part of the object.

need to supply a definition for that function (§ 6.1.2, p. 206). However, we must define every virtual function, regardless of whether it is used, because the compiler has no way to determine whether a virtual function is used.

Calls to Virtual Functions *May Be Resolved at Run Time*

When a virtual function is called through a reference or pointer, the compiler generates code to *decide at run time* which function to call. The function that is called is the one that corresponds to the dynamic type of the object bound to that pointer or reference.

As an example, consider our `print_total` function from § 15.1 (p. 593). That function calls `net_price` on its parameter named `item`, which has type `Quote&`. Because `item` is a reference, and because `net_price` is virtual, the version of `net_price` that is called depends at run time on the actual (dynamic) type of the argument bound to `item`:

```
Quote base("0-201-82470-1", 50);
print_total(cout, base, 10);    // calls Quote::net_price
Bulk_quote derived("0-201-82470-1", 50, 5, .19);
print_total(cout, derived, 10); // calls Bulk_quote::net_price
```

In the first call, `item` is bound to an object of type `Quote`. As a result, when

`print_total` calls `net_price`, the version defined by `Quote` is run. In the second call, `item` is bound to a `Bulk_quote` object. In this call, `print_total` calls the `Bulk_quote` version of `net_price`.

It is crucial to understand that dynamic binding happens only when a virtual function is called through a pointer or a reference.

```
base = derived;           // copies the Quote part of derived into base
base.net_price(20);       // calls Quote::net_price
```

When we call a virtual function on an expression that has a plain—nonreference and nonpointer—type, that call is bound at compile time. For example, when we call `net_price` on `base`, there is no question as to which version of `net_price` to run. We can change the value (i.e., the contents) of the object that `base` represents, but there is no way to change the type of that object. Hence, this call is resolved, at compile time, to the `Quote` version of `net_price`.

KEY CONCEPT: POLYMORPHISM IN C++

The key idea behind OOP is polymorphism. Polymorphism is derived from a Greek word meaning “many forms.” We speak of types related by inheritance as polymorphic types, because we can use the “many forms” of these types while ignoring the differences among them. The fact that the static and dynamic types of references and pointers can differ is the cornerstone of how C++ supports polymorphism.

When we call a function defined in a base class through a reference or pointer to the base class, we do not know the type of the object on which that member is executed. The object can be a base-class object or an object of a derived class. If the function is virtual, then the decision as to which function to run is delayed until run time. The version of the virtual function that is run is the one defined by the type of the object to which the reference is bound or to which the pointer points.

On the other hand, calls to nonvirtual functions are bound at compile time. Similarly, calls to any function (virtual or not) on an object are also bound at compile time. The type of an object is fixed and unvarying—there is nothing we can do to make the dynamic type of an object differ from its static type. Therefore, calls made on an object are bound at compile time to the version defined by the type of the object.



Virtinals are resolved at run time *only* if the call is made through a reference or pointer. Only in these cases is it possible for an object's dynamic type to differ from its static type.

Virtual Functions in a Derived Class

When a derived class overrides a virtual function, it may, but is not required to, repeat the `virtual` keyword. Once a function is declared as `virtual`, it remains `virtual` in all the derived classes.

A derived-class function that overrides an inherited virtual function must have exactly the same parameter type(s) as the base-class function that it overrides.

With one exception, the return type of a virtual in the derived class also must match the return type of the function from the base class. The exception applies to

virtuals that return a reference (or pointer) to types that are themselves related by inheritance. That is, if *D* is derived from *B*, then a base class virtual can return a *B** and the version in the derived can return a *D**. However, such return types require that the derived-to-base conversion from *D* to *B* is accessible. § 15.5 (p. 613) covers how to determine whether a base class is accessible. We'll see an example of this kind of virtual function in § 15.8.1 (p. 633).



A function that is virtual in a base class is implicitly virtual in its derived classes. When a derived class overrides a virtual, the parameters in the base and derived classes must match exactly.

The final and override Specifiers

As we'll see in § 15.6 (p. 620), it is legal for a derived class to define a function with the same name as a virtual in its base class but with a different parameter list. The compiler considers such a function to be independent from the base-class function. In such cases, the derived version does not override the version in the base class. In practice, such declarations often are a mistake—the class author intended to override a virtual from the base class but made a mistake in specifying the parameter list.

**C++
11**

Finding such bugs can be surprisingly hard. Under the new standard we can specify `override` on a virtual function in a derived class. Doing so makes our intention clear and (more importantly) enlists the compiler in finding such problems for us. The compiler will reject a program if a function marked `override` does not override an existing virtual function:

```
struct B {
    virtual void f1(int) const;
    virtual void f2();
    void f3();
};

struct D1 : B {
    void f1(int) const override; // ok: f1 matches f1 in the base
    void f2(int) override;      // error: B has no f2(int) function
    void f3() override;         // error: f3 not virtual
    void f4() override;         // error: B doesn't have a function named f4
};
```

In *D1*, the `override` specifier on *f1* is fine; both the base and derived versions of *f1* are `const` members that take an `int` and return `void`. The version of *f1* in *D1* properly overrides the virtual that it inherits from *B*.

The declaration of *f2* in *D1* does not match the declaration of *f2* in *B*—the version defined in *B* takes no arguments and the one defined in *D1* takes an `int`. Because the declarations don't match, *f2* in *D1* doesn't override *f2* from *B*; it is a new function that happens to have the same name. Because we said we intended this declaration to be an `override` and it isn't, the compiler will generate an error.

Because only a virtual function can be overridden, the compiler will also reject *f3* in *D1*. That function is not virtual in *B*, so there is no function to override.

Similarly `f4` is in error because `B` doesn't even have a function named `f4`.

We can also designate a function as `final`. Any attempt to override a function that has been defined as `final` will be flagged as an error:

```
struct D2 : B {
    // inherits f2() and f3() from B and overrides f1(int)
    void f1(int) const final; // subsequent classes can't override f1(int)
};

struct D3 : D2 {
    void f2(); // ok: overrides f2 inherited from the indirect base, B
    void f1(int) const; // error: D2 declared f2 as final
};
```

`final` and `override` specifiers appear after the parameter list (including any `const` or reference qualifiers) and after a trailing return (§ 6.3.3, p. 229).

Virtual Functions and Default Arguments

Like any other function, a virtual function can have default arguments (§ 6.5.1, p. 236). If a call uses a default argument, the value that is used is the one defined by the static type through which the function is called.

That is, when a call is made through a reference or pointer to base, the default argument(s) will be those defined in the base class. The base-class arguments will be used even when the derived version of the function is run. In this case, the derived function will be passed the default arguments defined for the base-class version of the function. If the derived function relies on being passed different arguments, the program will not execute as expected.

Best Practices

Virtual functions that have default arguments should use the same argument values in the base and derived classes.

Circumventing the Virtual Mechanism

In some cases, we want to prevent dynamic binding of a call to a virtual function; we want to force the call to use a particular version of that virtual. We can use the scope operator to do so. For example, this code:

```
// calls the version from the base class regardless of the dynamic type of baseP
double undiscounted = baseP->Quote::net_price(42);
```

calls the `Quote` version of `net_price` regardless of the type of the object to which `baseP` actually points. This call will be resolved at compile time.



Ordinarily, only code inside member functions (or friends) should need to use the scope operator to circumvent the virtual mechanism.

Why might we wish to circumvent the virtual mechanism? The most common reason is when a derived-class virtual function calls the version from the base class. In such cases, the base-class version might do work common to all types in the hierarchy. The versions defined in the derived classes would do whatever additional work is particular to their own type.

**WARNING**

If a derived virtual function that intended to call its base-class version omits the scope operator, the call will be resolved at run time as a call to the derived version itself, resulting in an infinite recursion.

EXERCISES SECTION 15.3

Exercise 15.11: Add a virtual debug function to your `Quote` class hierarchy that displays the data members of the respective classes.

Exercise 15.12: Is it ever useful to declare a member function as both `override` and `final`? Why or why not?

Exercise 15.13: Given the following classes, explain each `print` function:

```
class base {
public:
    string name() { return basename; }
    virtual void print(ostream &os) { os << basename; }
private:
    string basename;
};
class derived : public base {
public:
    void print(ostream &os) { print(os); os << " " << i; }
private:
    int i;
};
```

If there is a problem in this code, how would you fix it?

Exercise 15.14: Given the classes from the previous exercise and the following objects, determine which function is called at run time:

```
base bobj;      base *bp1 = &bobj;    base &br1 = bobj;
derived dobj;   base *bp2 = &dobj;    base &br2 = dobj;
(a) bobj.print(); (b) dobj.print(); (c) bp1->name();
(d) bp2->name(); (e) br1.print(); (f) br2.print();
```

15.4 Abstract Base Classes

Imagine that we want to extend our bookstore classes to support several discount strategies. In addition to a bulk discount, we might offer a discount for purchases up to a certain quantity and then charge the full price thereafter. Or we might offer a discount for purchases above a certain limit but not for purchases up to that limit.

Each of these discount strategies is the same in that it requires a quantity and a discount amount. We might support these differing strategies by defining a new class named `Disc_quote` to store the quantity and the discount amount. Classes, such as `Bulk_item`, that represent a specific discount strategy will inherit from

`Disc_quote`. Each of the derived classes will implement its discount strategy by defining its own version of `net_price`.

Before we can define our `Disc_Quote` class, we have to decide what to do about `net_price`. Our `Disc_quote` class doesn't correspond to any particular discount strategy; there is no meaning to ascribe to `net_price` for this class.

We could define `Disc_quote` without its own version of `net_price`. In this case, `Disc_quote` would inherit `net_price` from `Quote`.

However, this design would make it possible for our users to write nonsensical code. A user could create an object of type `Disc_quote` by supplying a quantity and a discount rate. Passing that `Disc_quote` object to a function such as `print_total` would use the `Quote` version of `net_price`. The calculated price would not include the discount that was supplied when the object was created. That state of affairs makes no sense.

Pure Virtual Functions

Thinking about the question in this detail reveals that our problem is not just that we don't know how to define `net_price`. In practice, we'd like to prevent users from creating `Disc_quote` objects at all. This class represents the general concept of a discounted book, not a concrete discount strategy.

We can enforce this design intent—and make it clear that there is no meaning for `net_price`—by defining `net_price` as a **pure virtual** function. Unlike ordinary virtuals, a pure virtual function does not have to be defined. We specify that a virtual function is a pure virtual by writing `= 0` in place of a function body (i.e., just before the semicolon that ends the declaration). The `= 0` may appear only on the declaration of a virtual function in the class body:

```
// class to hold the discount rate and quantity
// derived classes will implement pricing strategies using these data
class Disc_quote : public Quote {
public:
    Disc_quote() = default;
    Disc_quote(const std::string& book, double price,
               std::size_t qty, double disc):
        Quote(book, price),
        quantity(qty), discount(disc) { }
    double net_price(std::size_t) const = 0;
protected:
    std::size_t quantity = 0; // purchase size for the discount to apply
    double discount = 0.0;    // fractional discount to apply
};
```

Like our earlier `Bulk_item` class, `Disc_quote` defines a default constructor and a constructor that takes four parameters. Although we cannot define objects of this type directly, constructors in classes derived from `Disc_quote` will use the `Disc_quote` constructors to construct the `Disc_quote` part of their objects. The constructor that has four parameters passes its first two to the `Quote` constructor and directly initializes its own members, `discount` and `quantity`. The default constructor default initializes those members.

It is worth noting that we can provide a definition for a pure virtual. However, the function body must be defined outside the class. That is, we cannot provide a function body inside the class for a function that is = 0.

Classes with Pure Virtuals Are Abstract Base Classes

A class containing (or inheriting without overriding) a pure virtual function is an **abstract base class**. An abstract base class defines an interface for subsequent classes to override. We cannot (directly) create objects of a type that is an abstract base class. Because `Disc_quote` defines `net_price` as a pure virtual, we cannot define objects of type `Disc_quote`. We can define objects of classes that inherit from `Disc_quote`, so long as those classes override `net_price`:

```
// Disc_quote declares pure virtual functions, which Bulk_quote will override
Disc_quote discounted; // error: can't define a Disc_quote object
Bulk_quote bulk; // ok: Bulk_quote has no pure virtual functions
```

Classes that inherit from `Disc_quote` must define `net_price` or those classes will be abstract as well.



We may not create objects of a type that is an abstract base class.

A Derived Class Constructor Initializes Its Direct Base Class Only

Now we can reimplement `Bulk_quote` to inherit from `Disc_quote` rather than inheriting directly from `Quote`:

```
// the discount kicks in when a specified number of copies of the same book are sold
// the discount is expressed as a fraction to use to reduce the normal price
class Bulk_quote : public Disc_quote {
public:
    Bulk_quote() = default;
    Bulk_quote(const std::string& book, double price,
               std::size_t qty, double disc):
        Disc_quote(book, price, qty, disc) { }
    // overrides the base version to implement the bulk purchase discount policy
    double net_price(std::size_t) const override;
};
```

This version of `Bulk_quote` has a direct base class, `Disc_quote`, and an indirect base class, `Quote`. Each `Bulk_quote` object has three subobjects: an (empty) `Bulk_quote` part, a `Disc_quote` subobject, and a `Quote` subobject.

As we've seen, each class controls the initialization of objects of its type. Therefore, even though `Bulk_quote` has no data members of its own, it provides the same four-argument constructor as in our original class. Our new constructor passes its arguments to the `Disc_quote` constructor. That constructor in turn runs the `Quote` constructor. The `Quote` constructor initializes the `bookNo` and `price` members of `bulk`. When the `Quote` constructor ends, the `Disc_quote` constructor runs and initializes the `quantity` and `discount` members. At this

point, the `Bulk_quote` constructor resumes. That constructor has no further initializations or any other work to do.

KEY CONCEPT: REFACTORING

Adding `Disc_quote` to the `Quote` hierarchy is an example of *refactoring*. Refactoring involves redesigning a class hierarchy to move operations and/or data from one class to another. Refactoring is common in object-oriented applications.

It is noteworthy that even though we changed the inheritance hierarchy, code that uses `Bulk_quote` or `Quote` would not need to change. However, when classes are refactored (or changed in any other way) we must recompile any code that uses those classes.

EXERCISES SECTION 15.4

Exercise 15.15: Define your own versions of `Disc_quote` and `Bulk_quote`.

Exercise 15.16: Rewrite the class representing a limited discount strategy, which you wrote for the exercises in § 15.2.2 (p. 601), to inherit from `Disc_quote`.

Exercise 15.17: Try to define an object of type `Disc_quote` and see what errors you get from the compiler.

15.5 Access Control and Inheritance



Just as each class controls the initialization of its own members (§ 15.2.2, p. 598), each class also controls whether its members are **accessible** to a derived class.

protected Members

As we've seen, a class uses `protected` for those members that it is willing to share with its derived classes but wants to protect from general access. The `protected` specifier can be thought of as a blend of `private` and `public`:

- Like `private`, `protected` members are inaccessible to users of the class.
- Like `public`, `protected` members are accessible to members and friends of classes derived from this class.

In addition, `protected` has another important property:

- A derived class member or friend may access the `protected` members of the base class *only* through a derived object. The derived class has no special access to the `protected` members of base-class objects.

To understand this last rule, consider the following example:

```

class Base {
protected:
    int prot_mem;      // protected member
};
class Sneaky : public Base {
    friend void clobber(Sneaky&); // can access Sneaky::prot_mem
    friend void clobber(Base&);   // can't access Base::prot_mem
    int j;                        // j is private by default
};
// ok: clobber can access the private and protected members in Sneaky objects
void clobber(Sneaky &s) { s.j = s.prot_mem = 0; }
// error: clobber can't access the protected members in Base
void clobber(Base &b) { b.prot_mem = 0; }

```

If derived classes (and friends) could access protected members in a base-class object, then our second version of `clobber` (that takes a `Base&`) would be legal. That function is not a friend of `Base`, yet it would be allowed to change an object of type `Base`; we could circumvent the protection provided by `protected` for any class simply by defining a new class along the lines of `Sneaky`.

To prevent such usage, members and friends of a derived class can access the protected members *only* in base-class objects that are embedded inside a derived type object; they have no special access to ordinary objects of the base type.

public, private, and protected Inheritance

Access to a member that a class inherits is controlled by a combination of the access specifier for that member in the base class, and the access specifier in the derivation list of the derived class. As an example, consider the following hierarchy:

```

class Base {
public:
    void pub_mem(); // public member
protected:
    int prot_mem;   // protected member
private:
    char priv_mem;  // private member
};
struct Pub_Derv : public Base {
    // ok: derived classes can access protected members
    int f() { return prot_mem; }
    // error: private members are inaccessible to derived classes
    char g() { return priv_mem; }
};
struct Priv_Derv : private Base {
    // private derivation doesn't affect access in the derived class
    int fl() const { return prot_mem; }
};

```

The derivation access specifier has no effect on whether members (and friends) of a derived class may access the members of its own direct base class. Access to the

members of a base class is controlled by the access specifiers in the base class itself. Both `Pub_Derv` and `Priv_Derv` may access the protected member `prot_mem`. Neither may access the private member `priv_mem`.

The purpose of the derivation access specifier is to control the access that *users* of the derived class—including other classes derived from the derived class—have to the members inherited from `Base`:

```
Pub_Derv d1; // members inherited from Base are public
Priv_Derv d2; // members inherited from Base are private
d1.pub_mem(); // ok: pub_mem is public in the derived class
d2.pub_mem(); // error: pub_mem is private in the derived class
```

Both `Pub_Derv` and `Priv_Derv` inherit the `pub_mem` function. When the inheritance is public, members retain their access specification. Thus, `d1` can call `pub_mem`. In `Priv_Derv`, the members of `Base` are private; users of that class may not call `pub_mem`.

The derivation access specifier used by a derived class also controls access from classes that inherit from that derived class:

```
struct Derived_from_Public : public Pub_Derv {
    // ok: Base::prot_mem remains protected in Pub_Derv
    int use_base() { return prot_mem; }
};

struct Derived_from_Private : public Priv_Derv {
    // error: Base::prot_mem is private in Priv_Derv
    int use_base() { return prot_mem; }
};
```

Classes derived from `Pub_Derv` may access `prot_mem` from `Base` because that member remains a protected member in `Pub_Derv`. In contrast, classes derived from `Priv_Derv` have no such access. To them, all the members that `Priv_Derv` inherited from `Base` are private.

Had we defined another class, say, `Prot_Derv`, that used protected inheritance, the public members of `Base` would be protected members in that class. Users of `Prot_Derv` would have no access to `pub_mem`, but the members and friends of `Prot_Derv` could access that inherited member.

Accessibility of Derived-to-Base Conversion



Whether the derived-to-base conversion (§ 15.2.2, p. 597) is accessible depends on which code is trying to use the conversion and may depend on the access specifier used in the derived class' derivation. Assuming `D` inherits from `B`:

- User code may use the derived-to-base conversion *only* if `D` inherits publicly from `B`. User code may not use the conversion if `D` inherits from `B` using either protected or private.
- Member functions and friends of `D` can use the conversion to `B` regardless of how `D` inherits from `B`. The derived-to-base conversion to a direct base class is always accessible to members and friends of a derived class.

- Member functions and friends of classes derived from `D` may use the derived-to-base conversion if `D` inherits from `B` using either `public` or `protected`. Such code may not use the conversion if `D` inherits privately from `B`.



For any given point in your code, if a `public` member of the base class would be accessible, then the derived-to-base conversion is also accessible, and not otherwise.

KEY CONCEPT: CLASS DESIGN AND PROTECTED MEMBERS

In the absence of inheritance, we can think of a class as having two different kinds of users: ordinary users and implementors. Ordinary users write code that uses objects of the class type; such code can access only the `public` (interface) members of the class. Implementors write the code contained in the members and friends of the class. The members and friends of the class can access both the `public` and `private` (implementation) sections.

Under inheritance, there is a third kind of user, namely, derived classes. A base class makes `protected` those parts of its implementation that it is willing to let its derived classes use. The `protected` members remain inaccessible to ordinary user code; `private` members remain inaccessible to derived classes and their friends.

Like any other class, a class that is used as a base class makes its interface members `public`. A class that is used as a base class may divide its implementation into those members that are accessible to derived classes and those that remain accessible only to the base class and its friends. An implementation member should be `protected` if it provides an operation or data that a derived class will need to use in its own implementation. Otherwise, implementation members should be `private`.

Friendship and Inheritance

Just as friendship is not transitive (§ 7.3.4, p. 279), friendship is also not inherited. Friends of the base have no special access to members of its derived classes, and friends of a derived class have no special access to the base class:

```
class Base {
    // added friend declaration; other members as before
    friend class Pal; // Pal has no access to classes derived from Base
};
class Pal {
public:
    int f(Base b) { return b.prot_mem; } // ok: Pal is a friend of Base
    int f2(Sneaky s) { return s.j; } // error: Pal not friend of Sneaky
    // access to a base class is controlled by the base class, even inside a derived object
    int f3(Sneaky s) { return s.prot_mem; } // ok: Pal is a friend
};
```

The fact that `f3` is legal may seem surprising, but it follows directly from the notion that each class controls access to its own members. `Pal` is a friend of `Base`, so

Pal can access the members of Base objects. That access includes access to Base objects that are embedded in an object of a type derived from Base.

When a class makes another class a friend, it is only that class to which friendship is granted. The base classes of, and classes derived from, the friend have no special access to the befriending class:

```
// D2 has no access to protected or private members in Base
class D2 : public Pal {
public:
    int mem(Base b)
        { return b.prot_mem; } // error: friendship doesn't inherit
};
```



Friendship is not inherited; each class controls access to its members.

Exempting Individual Members

Sometimes we need to change the access level of a name that a derived class inherits. We can do so by providing a using declaration (§ 3.1, p. 82):

```
class Base {
public:
    std::size_t size() const { return n; }
protected:
    std::size_t n;
};

class Derived : private Base {    // note: private inheritance
public:
    // maintain access levels for members related to the size of the object
    using Base::size;
protected:
    using Base::n;
};
```

Because Derived uses private inheritance, the inherited members, `size` and `n`, are (by default) private members of Derived. The using declarations adjust the accessibility of these members. Users of Derived can access the `size` member, and classes subsequently derived from Derived can access `n`.

A using declaration inside a class can name any accessible (e.g., not private) member of a direct or indirect base class. Access to a name specified in a using declaration depends on the access specifier preceding the using declaration. That is, if a using declaration appears in a private part of the class, that name is accessible to members and friends only. If the declaration is in a public section, the name is available to all users of the class. If the declaration is in a protected section, the name is accessible to the members, friends, and derived classes.



A derived class may provide a using declaration only for names it is permitted to access.

Default Inheritance Protection Levels

In § 7.2 (p. 268) we saw that classes defined with the `struct` and `class` keywords have different default access specifiers. Similarly, the default derivation specifier depends on which keyword is used to define a derived class. By default, a derived class defined with the `class` keyword has `private` inheritance; a derived class defined with `struct` has `public` inheritance:

```
class Base { /* ... */ };
struct D1 : Base { /* ... */ }; // public inheritance by default
class D2 : Base { /* ... */ };  // private inheritance by default
```

It is a common misconception to think that there are deeper differences between classes defined using the `struct` keyword and those defined using `class`. The only differences are the default access specifier for members and the default derivation access specifier. There are no other distinctions.



A privately derived class should specify `private` explicitly rather than rely on the default. Being explicit makes it clear that private inheritance is intended and not an oversight.

EXERCISES SECTION 15.5

Exercise 15.18: Given the classes from page 612 and page 613, and assuming each object has the type specified in the comments, determine which of these assignments are legal. Explain why those that are illegal aren't allowed:

```
Base *p = &d1; // d1 has type Pub_Derv
p = &d2;      // d2 has type Priv_Derv
p = &d3;      // d3 has type Prot_Derv
p = &dd1;     // dd1 has type Derived_from_Public
p = &dd2;     // dd2 has type Derived_from_Private
p = &dd3;     // dd3 has type Derived_from_Protected
```

Exercise 15.19: Assume that each of the classes from page 612 and page 613 has a member function of the form:

```
void memfcn(Base &b) { b = *this; }
```

For each class, determine whether this function would be legal.

Exercise 15.20: Write code to test your answers to the previous two exercises.

Exercise 15.21: Choose one of the following general abstractions containing a family of types (or choose one of your own). Organize the types into an inheritance hierarchy:

- (a) Graphical file formats (such as gif, tiff, jpeg, bmp)
- (b) Geometric primitives (such as box, circle, sphere, cone)
- (c) C++ language types (such as class, function, member function)

Exercise 15.22: For the class you chose in the previous exercise, identify some of the likely virtual functions as well as `public` and `protected` members.

15.6 Class Scope under Inheritance



Each class defines its own scope (§ 7.4, p. 282) within which its members are defined. Under inheritance, the scope of a derived class is nested (§ 2.2.4, p. 48) inside the scope of its base classes. If a name is unresolved within the scope of the derived class, the enclosing base-class scopes are searched for a definition of that name.

The fact that the scope of a derived class nests inside the scope of its base classes can be surprising. After all, the base and derived classes are defined in separate parts of our program's text. However, it is this hierarchical nesting of class scopes that allows the members of a derived class to use members of its base class as if those members were part of the derived class. For example, when we write

```
Bulk_quote bulk;
cout << bulk.isbn();
```

the use of the name `isbn` is resolved as follows:

- Because we called `isbn` on an object of type `Bulk_quote`, the search starts in the `Bulk_quote` class. The name `isbn` is not found in that class.
- Because `Bulk_quote` is derived from `Disc_quote`, the `Disc_quote` class is searched next. The name is still not found.
- Because `Disc_quote` is derived from `Quote`, the `Quote` class is searched next. The name `isbn` is found in that class; the use of `isbn` is resolved to the `isbn` in `Quote`.

Name Lookup Happens at Compile Time

The static type (§ 15.2.3, p. 601) of an object, reference, or pointer determines which members of that object are visible. Even when the static and dynamic types might differ (as can happen when a reference or pointer to a base class is used), the static type determines what members can be used. As an example, we might add a member to the `Disc_quote` class that returns a pair (§ 11.2.3, p. 426) holding the minimum (or maximum) quantity and the discounted price:

```
class Disc_quote : public Quote {
public:
    std::pair<size_t, double> discount_policy() const
        { return {quantity, discount}; }
    // other members as before
};
```

We can use `discount_policy` only through an object, pointer, or reference of type `Disc_quote` or of a class derived from `Disc_quote`:

```
Bulk_quote bulk;
Bulk_quote *bulkP = &bulk; // static and dynamic types are the same
Quote *itemP = &bulk;      // static and dynamic types differ
bulkP->discount_policy();   // ok: bulkP has type Bulk_quote*
itemP->discount_policy();   // error: itemP has type Quote*
```

Even though `bulk` has a member named `discount_policy`, that member is not visible through `itemP`. The type of `itemP` is a pointer to `Quote`, which means that the search for `discount_policy` starts in class `Quote`. The `Quote` class has no member named `discount_policy`, so we cannot call that member on an object, reference, or pointer of type `Quote`.

Name Collisions and Inheritance

Like any other scope, a derived class can reuse a name defined in one of its direct or indirect base classes. As usual, names defined in an inner scope (e.g., a derived class) hide uses of that name in the outer scope (e.g., a base class) (§ 2.2.4, p. 48):

```
struct Base {
    Base(): mem(0) { }
protected:
    int mem;
};

struct Derived : Base {
    Derived(int i): mem(i) { } // initializes Derived::mem to i
                               // Base::mem is default initialized
    int get_mem() { return mem; } // returns Derived::mem
protected:
    int mem; // hides mem in the base
};
```

The reference to `mem` inside `get_mem` is resolved to the name inside `Derived`. Were we to write

```
Derived d(42);
cout << d.get_mem() << endl; // prints 42
```

then the output would be 42.



A derived-class member with the same name as a member of the base class hides direct use of the base-class member.

Using the Scope Operator to Use Hidden Members

We can use a hidden base-class member by using the scope operator:

```
struct Derived : Base {
    int get_base_mem() { return Base::mem; }
    // ...
};
```

The scope operator overrides the normal lookup and directs the compiler to look for `mem` starting in the scope of class `Base`. If we ran the code above with this version of `Derived`, the result of `d.get_mem()` would be 0.



Aside from overriding inherited virtual functions, a derived class usually should not reuse names defined in its base class.

KEY CONCEPT: NAME LOOKUP AND INHERITANCE

Understanding how function calls are resolved is crucial to understanding inheritance in C++. Given the call `p->mem()` (or `obj.mem()`), the following four steps happen:

- First determine the static type of `p` (or `obj`). Because we're calling a member, that type must be a class type.
- Look for `mem` in the class that corresponds to the static type of `p` (or `obj`). If `mem` is not found, look in the direct base class and continue up the chain of classes until `mem` is found or the last class is searched. If `mem` is not found in the class or its enclosing base classes, then the call will not compile.
- Once `mem` is found, do normal type checking (§ 6.1, p. 203) to see if this call is legal given the definition that was found.
- Assuming the call is legal, the compiler generates code, which varies depending on whether the call is virtual or not:
 - If `mem` is virtual and the call is made through a reference or pointer, then the compiler generates code to determine at run time which version to run based on the dynamic type of the object.
 - Otherwise, if the function is nonvirtual, or if the call is on an object (not a reference or pointer), the compiler generates a normal function call.

As Usual, Name Lookup Happens before Type Checking

As we've seen, functions declared in an inner scope do not overload functions declared in an outer scope (§ 6.4.1, p. 234). As a result, functions defined in a derived class do *not* overload members defined in its base class(es). As in any other scope, if a member in a derived class (i.e., in an inner scope) has the same name as a base-class member (i.e., a name defined in an outer scope), then the derived member hides the base-class member within the scope of the derived class. The base member is hidden even if the functions have different parameter lists:

```
struct Base {
    int memfcn();
};
struct Derived : Base {
    int memfcn(int);    // hides memfcn in the base
};
Derived d; Base b;
b.memfcn();            // calls Base::memfcn
d.memfcn(10);          // calls Derived::memfcn
d.memfcn();            // error: memfcn with no arguments is hidden
d.Base::memfcn();      // ok: calls Base::memfcn
```

The declaration of `memfcn` in `Derived` hides the declaration of `memfcn` in `Base`. Not surprisingly, the first call through `b`, which is a `Base` object, calls the version in the base class. Similarly, the second call (through `d`) calls the one from `Derived`. What can be surprising is that the third call, `d.memfcn()`, is illegal.

To resolve this call, the compiler looks for the name `memfcn` in `Derived`. That class defines a member named `memfcn` and the search stops. Once the name is found, the compiler looks no further. The version of `memfcn` in `Derived` expects an `int` argument. This call provides no such argument; it is in error.



Virtual Functions and Scope

We can now understand why virtual functions must have the same parameter list in the base and derived classes (§ 15.3, p. 605). If the base and derived members took arguments that differed from one another, there would be no way to call the derived version through a reference or pointer to the base class. For example:

```
class Base {
public:
    virtual int fcn();
};

class D1 : public Base {
public:
    // hides fcn in the base; this fcn is not virtual
    // D1 inherits the definition of Base::fcn()
    int fcn(int); // parameter list differs from fcn in Base
    virtual void f2(); // new virtual function that does not exist in Base
};

class D2 : public D1 {
public:
    int fcn(int); // nonvirtual function hides D1::fcn(int)
    int fcn(); // overrides virtual fcn from Base
    void f2(); // overrides virtual f2 from D1
};
```

The `fcn` function in `D1` does not override the virtual `fcn` from `Base` because they have different parameter lists. Instead, it *hides* `fcn` from the base. Effectively, `D1` has two functions named `fcn`: `D1` inherits a virtual named `fcn` from `Base` and defines its own, nonvirtual member named `fcn` that takes an `int` parameter.

Calling a Hidden Virtual through the Base Class

Given the classes above, let's look at several different ways to call these functions:

```
Base bobj; D1 d1obj; D2 d2obj;

Base *bp1 = &bobj, *bp2 = &d1obj, *bp3 = &d2obj;
bp1->fcn(); // virtual call, will call Base::fcn at run time
bp2->fcn(); // virtual call, will call Base::fcn at run time
bp3->fcn(); // virtual call, will call D2::fcn at run time

D1 *d1p = &d1obj; D2 *d2p = &d2obj;
bp2->f2(); // error: Base has no member named f2
d1p->f2(); // virtual call, will call D1::f2() at run time
d2p->f2(); // virtual call, will call D2::f2() at run time
```

The first three calls are all made through pointers to the base class. Because `fcn` is virtual, the compiler generates code to decide at run time which version to call.

That decision will be based on the actual type of the object to which the pointer is bound. In the case of `bp2`, the underlying object is a `D1`. That class did not override the `fcn` function that takes no arguments. Thus, the call through `bp2` is resolved (at run time) to the version defined in `Base`.

The next three calls are made through pointers with differing types. Each pointer points to one of the types in this hierarchy. The first call is illegal because there is no `f2()` in class `Base`. The fact that the pointer happens to point to a derived object is irrelevant.

For completeness, let's look at calls to the nonvirtual function `fcn(int)`:

```
Base *p1 = &d2obj; D1 *p2 = &d2obj; D2 *p3 = &d2obj;
p1->fcn(42);    // error: Base has no version of fcn that takes an int
p2->fcn(42);    // statically bound, calls D1::fcn(int)
p3->fcn(42);    // statically bound, calls D2::fcn(int)
```

In each call the pointer happens to point to an object of type `D2`. However, the dynamic type doesn't matter when we call a nonvirtual function. The version that is called depends only on the static type of the pointer.

Overriding Overloaded Functions

As with any other function, a member function (virtual or otherwise) can be overloaded. A derived class can override zero or more instances of the overloaded functions it inherits. If a derived class wants to make all the overloaded versions available through its type, then it must override all of them or none of them.

Sometimes a class needs to override some, but not all, of the functions in an overloaded set. It would be tedious in such cases to have to override every base-class version in order to override the ones that the class needs to specialize.

Instead of overriding every base-class version that it inherits, a derived class can provide a `using` declaration (§ 15.5, p. 615) for the overloaded member. A `using` declaration specifies only a name; it may not specify a parameter list. Thus, a `using` declaration for a base-class member function adds all the overloaded instances of that function to the scope of the derived class. Having brought all the names into its scope, the derived class needs to define only those functions that truly depend on its type. It can use the inherited definitions for the others.

The normal rules for a `using` declaration inside a class apply to names of overloaded functions (§ 15.5, p. 615); every overloaded instance of the function in the base class must be accessible to the derived class. The access to the overloaded versions that are not otherwise redefined by the derived class will be the access in effect at the point of the `using` declaration.

EXERCISES SECTION 15.6

Exercise 15.23: Assuming class `D1` on page 620 had intended to override its inherited `fcn` function, how would you fix that class? Assuming you fixed the class so that `fcn` matched the definition in `Base`, how would the calls in that section be resolved?

15.7 Constructors and Copy Control

Like any other class, a class in an inheritance hierarchy controls what happens when objects of its type are created, copied, moved, assigned, or destroyed. As for any other class, if a class (base or derived) does not itself define one of the copy-control operations, the compiler will synthesize that operation. Also, as usual, the synthesized version of any of these members might be a deleted function.



15.7.1 Virtual Destructors

The primary direct impact that inheritance has on copy control for a base class is that a base class generally should define a virtual destructor (§ 15.2.1, p. 594). The destructor needs to be virtual to allow objects in the inheritance hierarchy to be dynamically allocated.

Recall that the destructor is run when we delete a pointer to a dynamically allocated object (§ 13.1.3, p. 502). If that pointer points to a type in an inheritance hierarchy, it is possible that the static type of the pointer might differ from the dynamic type of the object being destroyed (§ 15.2.2, p. 597). For example, if we delete a pointer of type `Quote*`, that pointer might point at a `Bulk_quote` object. If the pointer points at a `Bulk_quote`, the compiler has to know that it should run the `Bulk_quote` destructor. As with any other function, we arrange to run the proper destructor by defining the destructor as virtual in the base class:

```
class Quote {
public:
    // virtual destructor needed if a base pointer pointing to a derived object is deleted
    virtual ~Quote() = default; // dynamic binding for the destructor
};
```

Like any other virtual, the virtual nature of the destructor is inherited. Thus, classes derived from `Quote` have virtual destructors, whether they use the synthesized destructor or define their own version. So long as the base class destructor is virtual, when we delete a pointer to base, the correct destructor will be run:

```
Quote *itemP = new Quote; // same static and dynamic type
delete itemP;             // destructor for Quote called
itemP = new Bulk_quote;   // static and dynamic types differ
delete itemP;             // destructor for Bulk_quote called
```



WARNING

Executing `delete` on a pointer to base that points to a derived object has undefined behavior if the base's destructor is not virtual.

Destructors for base classes are an important exception to the rule of thumb that if a class needs a destructor, it also needs copy and assignment (§ 13.1.4, p. 504). A base class almost always needs a destructor, so that it can make the destructor virtual. If a base class has an empty destructor in order to make it virtual, then the fact that the class has a destructor does not indicate that the assignment operator or copy constructor is also needed.

Virtual Destructors Turn Off Synthesized Move

The fact that a base class needs a virtual destructor has an important indirect impact on the definition of base and derived classes: If a class defines a destructor—even if it uses `= default` to use the synthesized version—the compiler will not synthesize a move operation for that class (§ 13.6.2, p. 537).

EXERCISES SECTION 15.7.1

Exercise 15.24: What kinds of classes need a virtual destructor? What operations must a virtual destructor perform?

15.7.2 Synthesized Copy Control and Inheritance



The synthesized copy-control members in a base or a derived class execute like any other synthesized constructor, assignment operator, or destructor: They memberwise initialize, assign, or destroy the members of the class itself. In addition, these synthesized members initialize, assign, or destroy the direct base part of an object by using the corresponding operation from the base class. For example,

- The synthesized `Bulk_quote` default constructor runs the `Disc_quote` default constructor, which in turn runs the `Quote` default constructor.
- The `Quote` default constructor default initializes the `bookNo` member to the empty string and uses the in-class initializer to initialize `price` to zero.
- When the `Quote` constructor finishes, the `Disc_quote` constructor continues, which uses the in-class initializers to initialize `qty` and `discount`.
- When the `Disc_quote` constructor finishes, the `Bulk_quote` constructor continues but has no other work to do.

Similarly, the synthesized `Bulk_quote` copy constructor uses the (synthesized) `Disc_quote` copy constructor, which uses the (synthesized) `Quote` copy constructor. The `Quote` copy constructor copies the `bookNo` and `price` members; and the `Disc_quote` copy constructor copies the `qty` and `discount` members.

It is worth noting that it doesn't matter whether the base-class member is itself synthesized (as is the case in our `Quote` hierarchy) or has a user-provided definition. All that matters is that the corresponding member is accessible (§ 15.5, p. 611) and that it is not a deleted function.

Each of our `Quote` classes use the synthesized destructor. The derived classes do so implicitly, whereas the `Quote` class does so explicitly by defining its (virtual) destructor as `= default`. The synthesized destructor is (as usual) empty and its implicit destruction part destroys the members of the class (§ 13.1.3, p. 501). In addition to destroying its own members, the destruction phase of a destructor in a derived class also destroys its direct base. That destructor in turn invokes the destructor for its own direct base, if any. And, so on up to the root of the hierarchy.

As we've seen, `Quote` does not have synthesized move operations because it defines a destructor. The (synthesized) copy operations will be used whenever we move a `Quote` object (§ 13.6.2, p. 540). As we're about to see, the fact that `Quote` does not have move operations means that its derived classes don't either.

Base Classes and Deleted Copy Control in the Derived

**C++
11**

The synthesized default constructor, or any of the copy-control members of either a base or a derived class, may be defined as deleted for the same reasons as in any other class (§ 13.1.6, p. 508, and § 13.6.2, p. 537). In addition, the way in which a base class is defined can cause a derived-class member to be defined as deleted:

- If the default constructor, copy constructor, copy-assignment operator, or destructor in the base class is deleted or inaccessible (§ 15.5, p. 612), then the corresponding member in the derived class is defined as deleted, because the compiler can't use the base-class member to construct, assign, or destroy the base-class part of the object.
- If the base class has an inaccessible or deleted destructor, then the synthesized default and copy constructors in the derived classes are defined as deleted, because there is no way to destroy the base part of the derived object.
- As usual, the compiler will not synthesize a deleted move operation. If we use `= default` to request a move operation, it will be a deleted function in the derived if the corresponding operation in the base is deleted or inaccessible, because the base class part cannot be moved. The move constructor will also be deleted if the base class destructor is deleted or inaccessible.

As an example, this base class, `B`,

```
class B {
public:
    B();
    B(const B&) = delete;
    // other members, not including a move constructor
};
class D : public B {
    // no constructors
};
D d;           // ok: D's synthesized default constructor uses B's default constructor
D d2(d);      // error: D's synthesized copy constructor is deleted
D d3(std::move(d)); // error: implicitly uses D's deleted copy constructor
```

has an accessible default constructor and an explicitly deleted copy constructor. Because the copy constructor is defined, the compiler will not synthesize a move constructor for class `B` (§ 13.6.2, p. 537). As a result, we can neither move nor copy objects of type `B`. If a class derived from `B` wanted to allow its objects to be copied or moved, that derived class would have to define its own versions of these constructors. Of course, that class would have to decide how to copy or move the members in its base-class part. In practice, if a base class does not have a default, copy, or move constructor, then its derived classes usually don't either.

Move Operations and Inheritance

As we've seen, most base classes define a virtual destructor. As a result, by default, base classes generally do not get synthesized move operations. Moreover, by default, classes derived from a base class that doesn't have move operations don't get synthesized move operations either.

Because lack of a move operation in a base class suppresses synthesized move for its derived classes, base classes ordinarily should define the move operations if it is sensible to do so. Our `Quote` class can use the synthesized versions. However, `Quote` must define these members explicitly. Once it defines its move operations, it must also explicitly define the copy versions as well (§ 13.6.2, p. 539):

```
class Quote {
public:
    Quote() = default;           // memberwise default initialize
    Quote(const Quote&) = default; // memberwise copy
    Quote(Quote&&) = default;     // memberwise copy
    Quote& operator=(const Quote&) = default; // copy assign
    Quote& operator=(Quote&&) = default;      // move assign
    virtual ~Quote() = default;
    // other members as before
};
```

Now, `Quote` objects will be memberwise copied, moved, assigned, and destroyed. Moreover, classes derived from `Quote` will automatically obtain synthesized move operations as well, unless they have members that otherwise preclude move.

EXERCISES SECTION 15.7.2

Exercise 15.25: Why did we define a default constructor for `Disc_quote`? What effect, if any, would removing that constructor have on the behavior of `Bulk_quote`?

15.7.3 Derived-Class Copy-Control Members



As we saw in § 15.2.2 (p. 598), the initialization phase of a derived-class constructor initializes the base-class part(s) of a derived object as well as initializing its own members. As a result, the copy and move constructors for a derived class must copy/move the members of its base part as well as the members in the derived. Similarly, a derived-class assignment operator must assign the members in the base part of the derived object.

Unlike the constructors and assignment operators, the destructor is responsible only for destroying the resources allocated by the derived class. Recall that the members of an object are implicitly destroyed (§ 13.1.3, p. 502). Similarly, the base-class part of a derived object is destroyed automatically.



When a derived class defines a copy or move operation, that operation is responsible for copying or moving the entire object, including base-class members.



Defining a Derived Copy or Move Constructor

When we define a copy or move constructor (§ 13.1.1, p. 496, and § 13.6.2, p. 534) for a derived class, we ordinarily use the corresponding base-class constructor to initialize the base part of the object:

```
class Base { /* ... */ };
class D: public Base {
public:
    // by default, the base class default constructor initializes the base part of an object
    // to use the copy or move constructor, we must explicitly call that
    // constructor in the constructor initializer list
    D(const D& d): Base(d)           // copy the base members
                /* initializers for members of D */ { /* ... */ }
    D(D&& d): Base(std::move(d)) // move the base members
                /* initializers for members of D */ { /* ... */ }
};
```

The initializer `Base(d)` passes a `D` object to a base-class constructor. Although in principle, `Base` could have a constructor that has a parameter of type `D`, in practice, that is very unlikely. Instead, `Base(d)` will (ordinarily) match the `Base` copy constructor. The `D` object, `d`, will be bound to the `Base&` parameter in that constructor. The `Base` copy constructor will copy the base part of `d` into the object that is being created. Had the initializer for the base class been omitted,

```
// probably incorrect definition of the D copy constructor
// base-class part is default initialized, not copied
D(const D& d) /* member initializers, but no base-class initializer */
{ /* ... */ }
```

the `Base` default constructor would be used to initialize the base part of a `D` object. Assuming `D`'s constructor copies the derived members from `d`, this newly constructed object would be oddly configured: Its `Base` members would hold default values, while its `D` members would be copies of the data from another object.



By default, the base-class default constructor initializes the base-class part of a derived object. If we want copy (or move) the base-class part, we must explicitly use the copy (or move) constructor for the base class in the derived's constructor initializer list.

Derived-Class Assignment Operator

Like the copy and move constructors, a derived-class assignment operator (§ 13.1.2, p. 500, and § 13.6.2, p. 536), must assign its base part explicitly:

```
// Base::operator=(const Base&) is not invoked automatically
D &D::operator=(const D &rhs)
{
    Base::operator=(rhs); // assigns the base part
    // assign the members in the derived class, as usual,
    // handling self-assignment and freeing existing resources as appropriate
    return *this;
}
```

This operator starts by explicitly calling the base-class assignment operator to assign the members of the base part of the derived object. The base-class operator will (presumably) correctly handle self-assignment and, if appropriate, will free the old value in the base part of the left-hand operand and assign the new values from rhs. Once that operator finishes, we continue doing whatever is needed to assign the members in the derived class.

It is worth noting that a derived constructor or assignment operator can use its corresponding base class operation regardless of whether the base defined its own version of that operator or uses the synthesized version. For example, the call to `Base::operator=` executes the copy-assignment operator in class `Base`. It is immaterial whether that operator is defined explicitly by the `Base` class or is synthesized by the compiler.

Derived-Class Destructor

Recall that the data members of an object are implicitly destroyed after the destructor body completes (§ 13.1.3, p. 502). Similarly, the base-class parts of an object are also implicitly destroyed. As a result, unlike the constructors and assignment operators, a derived destructor is responsible only for destroying the resources allocated by the derived class:

```
class D: public Base {
public:
    // Base::~~Base invoked automatically
    ~D() { /* do what it takes to clean up derived members */ }
};
```

Objects are destroyed in the opposite order from which they are constructed: The derived destructor is run first, and then the base-class destructors are invoked, back up through the inheritance hierarchy.

Calls to Virtuals in Constructors and Destructors

As we've seen, the base-class part of a derived object is constructed first. While the base-class constructor is executing, the derived part of the object is uninitialized. Similarly, derived objects are destroyed in reverse order, so that when a base class destructor runs, the derived part has already been destroyed. As a result, while these base-class members are executing, the object is incomplete.

To accommodate this incompleteness, the compiler treats the object as if its type changes during construction or destruction. That is, while an object is being constructed it is treated as if it has the same class as the constructor; calls to virtual

functions will be bound as if the object has the same type as the constructor itself. Similarly, for destructors. This binding applies to virtuals called directly or that are called indirectly from a function that the constructor (or destructor) calls.

To understand this behavior, consider what would happen if the derived-class version of a virtual was called from a base-class constructor. This virtual probably accesses members of the derived object. After all, if the virtual didn't need to use members of the derived object, the derived class probably could use the version in its base class. However, those members are uninitialized while a base constructor is running. If such access were allowed, the program would probably crash.



If a constructor or destructor calls a virtual, the version that is run is the one corresponding to the type of the constructor or destructor itself.

EXERCISES SECTION 15.7.3

Exercise 15.26: Define the `Quote` and `Bulk_quote` copy-control members to do the same job as the synthesized versions. Give them and the other constructors print statements that identify which function is running. Write programs using these classes and predict what objects will be created and destroyed. Compare your predictions with the output and continue experimenting until your predictions are reliably correct.

15.7.4 Inherited Constructors

C++
11

Under the new standard, a derived class can reuse the constructors defined by its direct base class. Although, as we'll see, such constructors are not inherited in the normal sense of that term, it is nonetheless common to refer to such constructors as "inherited." For the same reasons that a class may initialize only its direct base class, a class may inherit constructors only from its direct base. A class cannot inherit the default, copy, and move constructors. If the derived class does not directly define these constructors, the compiler synthesizes them as usual.

A derived class inherits its base-class constructors by providing a `using` declaration that names its (direct) base class. As an example, we can redefine our `Bulk_quote` class (§ 15.4, p. 610) to inherit its constructors from `Disc_quote`:

```
class Bulk_quote : public Disc_quote {
public:
    using Disc_quote::Disc_quote; // inherit Disc_quote's constructors
    double net_price(std::size_t) const;
};
```

Ordinarily, a `using` declaration only makes a name visible in the current scope. When applied to a constructor, a `using` declaration causes the compiler to generate code. The compiler generates a derived constructor corresponding to each constructor in the base. That is, for each constructor in the base class, the compiler generates a constructor in the derived class that has the same parameter list.

These compiler-generated constructors have the form

```
derived(parms) : base(args) { }
```

where *derived* is the name of the derived class, *base* is the name of the base class, *parms* is the parameter list of the constructor, and *args* pass the parameters from the derived constructor to the base constructor. In our `Bulk_quote` class, the inherited constructor would be equivalent to

```
Bulk_quote(const std::string& book, double price,
           std::size_t qty, double disc):
    Disc_quote(book, price, qty, disc) { }
```

If the derived class has any data members of its own, those members are default initialized (§ 7.1.4, p. 266).

Characteristics of an Inherited Constructor

Unlike using declarations for ordinary members, a constructor using declaration does not change the access level of the inherited constructor(s). For example, regardless of where the using declaration appears, a `private` constructor in the base is a `private` constructor in the derived; similarly for `protected` and `public` constructors.

Moreover, a using declaration can't specify `explicit` or `constexpr`. If a constructor in the base is `explicit` (§ 7.5.4, p. 296) or `constexpr` (§ 7.5.6, p. 299), the inherited constructor has the same property.

If a base-class constructor has default arguments (§ 6.5.1, p. 236), those arguments are not inherited. Instead, the derived class gets multiple inherited constructors in which each parameter with a default argument is successively omitted. For example, if the base has a constructor with two parameters, the second of which has a default, the derived class will obtain two constructors: one with both parameters (and no default argument) and a second constructor with a single parameter corresponding to the left-most, non-defaulted parameter in the base class.

If a base class has several constructors, then with two exceptions, the derived class inherits each of the constructors from its base class. The first exception is that a derived class can inherit some constructors and define its own versions of other constructors. If the derived class defines a constructor with the same parameters as a constructor in the base, then that constructor is not inherited. The one defined in the derived class is used in place of the inherited constructor.

The second exception is that the default, copy, and move constructors are not inherited. These constructors are synthesized using the normal rules. An inherited constructor is not treated as a user-defined constructor. Therefore, a class that contains only inherited constructors will have a synthesized default constructor.

EXERCISES SECTION 15.7.4

Exercise 15.27: Redefine your `Bulk_quote` class to inherit its constructors.



15.8 Containers and Inheritance

When we use a container to store objects from an inheritance hierarchy, we generally must store those objects indirectly. We cannot put objects of types related by inheritance directly into a container, because there is no way to define a container that holds elements of differing types.

As an example, assume we want to define a `vector` to hold several books that a customer wants to buy. It should be easy to see that we can't use a `vector` that holds `Bulk_quote` objects. We can't convert `Quote` objects to `Bulk_quote` (§ 15.2.3, p. 602), so we wouldn't be able to put `Quote` objects into that `vector`.

It may be somewhat less obvious that we also can't use a `vector` that holds objects of type `Quote`. In this case, we can put `Bulk_quote` objects into the container. However, those objects would no longer be `Bulk_quote` objects:

```
vector<Quote> basket;
basket.push_back(Quote("0-201-82470-1", 50));
// ok, but copies only the Quote part of the object into basket
basket.push_back(Bulk_quote("0-201-54848-8", 50, 10, .25));
// calls version defined by Quote, prints 750, i.e., 15 * $50
cout << basket.back().net_price(15) << endl;
```

The elements in `basket` are `Quote` objects. When we add a `Bulk_quote` object to the `vector` its derived part is ignored (§ 15.2.3, p. 603).



WARNING

Because derived objects are “sliced down” when assigned to a base-type object, containers and types related by inheritance do not mix well.

Put (Smart) Pointers, Not Objects, in Containers

When we need a container that holds objects related by inheritance, we typically define the container to hold pointers (preferably smart pointers (§ 12.1, p. 450)) to the base class. As usual, the dynamic type of the object to which those pointers point might be the base-class type or a type derived from that base:

```
vector<shared_ptr<Quote>> basket;
basket.push_back(make_shared<Quote>("0-201-82470-1", 50));
basket.push_back(
    make_shared<Bulk_quote>("0-201-54848-8", 50, 10, .25));
// calls the version defined by Quote; prints 562.5, i.e., 15 * $50 less the discount
cout << basket.back()->net_price(15) << endl;
```

Because `basket` holds `shared_ptr`s, we must dereference the value returned by `basket.back()` to get the object on which to run `net_price`. We do so by using `->` in the call to `net_price`. As usual, the version of `net_price` that is called depends on the dynamic type of the object to which that pointer points.

It is worth noting that we defined `basket` as `shared_ptr<Quote>`, yet in the second `push_back` we passed a `shared_ptr` to a `Bulk_quote` object. Just as we can convert an ordinary pointer to a derived type to a pointer to an base-class type (§ 15.2.2, p. 597), we can also convert a smart pointer to a derived type to a

smart pointer to an base-class type. Thus, `make_shared<Bulk_quote>` returns a `shared_ptr<Bulk_quote>` object, which is converted to `shared_ptr<Quote>` when we call `push_back`. As a result, despite appearances, all of the elements of `basket` have the same type.

EXERCISES SECTION 15.8

Exercise 15.28: Define a vector to hold `Quote` objects but put `Bulk_quote` objects into that vector. Compute the total `net_price` of all the elements in the vector.

Exercise 15.29: Repeat your program, but this time store `shared_ptr`s to objects of type `Quote`. Explain any discrepancy in the sum generated by the this version and the previous program. If there is no discrepancy, explain why there isn't one.

15.8.1 Writing a Basket Class



One of the ironies of object-oriented programming in C++ is that we cannot use objects directly to support it. Instead, we must use pointers and references. Because pointers impose complexity on our programs, we often define auxiliary classes to help manage that complexity. We'll start by defining a class to represent a basket:

```
class Basket {
public:
    // Basket uses synthesized default constructor and copy-control members
    void add_item(const std::shared_ptr<Quote> &sale)
    { items.insert(sale); }
    // prints the total price for each book and the overall total for all items in the basket
    double total_receipt(std::ostream&) const;
private:
    // function to compare shared_ptrs needed by the multiset member
    static bool compare(const std::shared_ptr<Quote> &lhs,
                       const std::shared_ptr<Quote> &rhs)
    { return lhs->isbn() < rhs->isbn(); }
    // multiset to hold multiple quotes, ordered by the compare member
    std::multiset<std::shared_ptr<Quote>, decltype(compare)*>
        items{compare};
};
```

Our class uses a `multiset` (§ 11.2.1, p. 423) to hold the transactions, so that we can store multiple transactions for the same book, and so that all the transactions for a given book will be kept together (§ 11.2.2, p. 424).

The elements in our `multiset` are `shared_ptr`s and there is no less-than operator for `shared_ptr`. As a result, we must provide our own comparison operation to order the elements (§ 11.2.2, p. 425). Here, we define a private static member, named `compare`, that compares the `isbn`s of the objects to which the `shared_ptr`s point. We initialize our `multiset` to use this comparison function through an in-class initializer (§ 7.3.1, p. 274):

```
// multiset to hold multiple quotes, ordered by the compare member
std::multiset<std::shared_ptr<Quote>, decltype(compare)*>
    items{compare};
```

This declaration can be hard to read, but reading from left to right, we see that we are defining a multiset of `shared_ptr`s to `Quote` objects. The multiset will use a function with the same type as our `compare` member to order the elements. The multiset member is named `items`, and we're initializing `items` to use our `compare` function.

Defining the Members of `Basket`

The `Basket` class defines only two operations. We defined the `add_item` member inside the class. That member takes a `shared_ptr` to a dynamically allocated `Quote` and puts that `shared_ptr` into the multiset. The second member, `total_receipt`, prints an itemized bill for the contents of the basket and returns the price for all the items in the basket:

```
double Basket::total_receipt(ostream &os) const
{
    double sum = 0.0;    // holds the running total

    // iter refers to the first element in a batch of elements with the same ISBN
    // upper_bound returns an iterator to the element just past the end of that batch
    for (auto iter = items.cbegin();
         iter != items.cend();
         iter = items.upper_bound(*iter)) {
        // we know there's at least one element with this key in the Basket
        // print the line item for this book
        sum += print_total(os, **iter, items.count(*iter));
    }
    os << "Total Sale: " << sum << endl; // print the final overall total
    return sum;
}
```

Our `for` loop starts by defining and initializing `iter` to refer to the first element in the multiset. The condition checks whether `iter` is equal to `items.cend()`. If so, we've processed all the purchases and we drop out of the `for`. Otherwise, we process the next book.

The interesting bit is the “increment” expression in the `for`. Rather than the usual loop that reads each element, we advance `iter` to refer to the next key. We skip over all the elements that match the current key by calling `upper_bound` (§ 11.3.5, p. 438). The call to `upper_bound` returns the iterator that refers to the element just past the last one with the same key as in `iter`. The iterator we get back denotes either the end of the set or the next book.

Inside the `for` loop, we call `print_total` (§ 15.1, p. 593) to print the details for each book in the basket:

```
sum += print_total(os, **iter, items.count(*iter));
```

The arguments to `print_total` are an `ostream` on which to write, a `Quote` object to process, and a count. When we dereference `iter`, we get a `shared_ptr`

that points to the object we want to print. To get that object, we must dereference that `shared_ptr`. Thus, `**iter` is a `Quote` object (or an object of a type derived from `Quote`). We use the `multiset` count member (§ 11.3.5, p. 436) to determine how many elements in the `multiset` have the same key (i.e., the same ISBN).

As we've seen, `print_total` makes a virtual call to `net_price`, so the resulting price depends on the dynamic type of `**iter`. The `print_total` function prints the total for the given book and returns the total price that it calculated. We add that result into `sum`, which we print after we complete the `for` loop.

Hiding the Pointers

Users of `Basket` still have to deal with dynamic memory, because `add_item` takes a `shared_ptr`. As a result, users have to write code such as

```
Basket bsk;
bsk.add_item(make_shared<Quote>("123", 45));
bsk.add_item(make_shared<Bulk_quote>("345", 45, 3, .15));
```

Our next step will be to redefine `add_item` so that it takes a `Quote` object instead of a `shared_ptr`. This new version of `add_item` will handle the memory allocation so that our users no longer need to do so. We'll define two versions, one that will copy its given object and the other that will move from it (§ 13.6.3, p. 544):

```
void add_item(const Quote& sale); // copy the given object
void add_item(Quote&& sale);      // move the given object
```

The only problem is that `add_item` doesn't know what type to allocate. When it does its memory allocation, `add_item` will copy (or move) its `sale` parameter. Somewhere there will be a new expression such as:

```
new Quote(sale)
```

Unfortunately, this expression won't do the right thing: `new` allocates an object of the type we request. This expression allocates an object of type `Quote` and copies the `Quote` portion of `sale`. However, `sale` might refer to a `Bulk_quote` object, in which case, that object will be sliced down.

Simulating Virtual Copy



We'll solve this problem by giving our `Quote` classes a virtual member that allocates a copy of itself.

```
class Quote {
public:
    // virtual function to return a dynamically allocated copy of itself
    // these members use reference qualifiers; see § 13.6.3 (p. 546)
    virtual Quote* clone() const & {return new Quote(*this);}
    virtual Quote* clone() &&
                                {return new Quote(std::move(*this));}
    // other members as before
};
```

```

class Bulk_quote : public Quote {
    Bulk_quote* clone() const & {return new Bulk_quote(*this);}
    Bulk_quote* clone() &&
        {return new Bulk_quote(std::move(*this));}
    // other members as before
};

```

Because we have a copy and a move version of `add_item`, we defined lvalue and rvalue versions of `clone` (§ 13.6.3, p. 546). Each `clone` function allocates a new object of its own type. The `const` lvalue reference member copies itself into that newly allocated object; the rvalue reference member moves its own data.

Using `clone`, it is easy to write our new versions of `add_item`:

```

class Basket {
public:
    void add_item(const Quote& sale) // copy the given object
    { items.insert(std::shared_ptr<Quote>(sale.clone())); }
    void add_item(Quote&& sale) // move the given object
    { items.insert(
        std::shared_ptr<Quote>(std::move(sale).clone())); }
    // other members as before
};

```

Like `add_item` itself, `clone` is overloaded based on whether it is called on an lvalue or an rvalue. Thus, the first version of `add_item` calls the `const` lvalue version of `clone`, and the second version calls the rvalue reference version. Note that in the rvalue version, although the type of `sale` is an rvalue reference type, `sale` (like any other variable) is an lvalue (§ 13.6.1, p. 533). Therefore, we call `move` to bind an rvalue reference to `sale`.

Our `clone` function is also virtual. Whether the `Quote` or `Bulk_quote` function is run, depends (as usual) on the dynamic type of `sale`. Regardless of whether we copy or move the data, `clone` returns a pointer to a newly allocated object, of its own type. We bind a `shared_ptr` to that object and call `insert` to add this newly allocated object to `items`. Note that because `shared_ptr` supports the derived-to-base conversion (§ 15.2.2, p. 597), we can bind a `shared_ptr<Quote>` to a `Bulk_quote*`.

EXERCISES SECTION 15.8.1

Exercise 15.30: Write your own version of the `Basket` class and use it to compute prices for the same transactions as you used in the previous exercises.

15.9 Text Queries Revisited

As a final example of inheritance, we'll extend our text-query application from § 12.3 (p. 484). The classes we wrote in that section let us look for occurrences of a

given word in a file. We'd like to extend the system to support more complicated queries. In our examples, we'll run queries against the following simple story:

```
Alice Emma has long flowing red hair.
Her Daddy says when the wind blows
through her hair, it looks almost alive,
like a fiery bird in flight.
A beautiful fiery bird, he tells her,
magical but untamed.
"Daddy, shush, there is no such thing,"
she tells him, at the same time wanting
him to tell her more.
Shyly, she asks, "I mean, Daddy, is there?"
```

Our system should support the following queries:

- Word queries find all the lines that match a given string:

```
Executing Query for: Daddy
Daddy occurs 3 times
(line 2) Her Daddy says when the wind blows
(line 7) "Daddy, shush, there is no such thing,"
(line 10) Shyly, she asks, "I mean, Daddy, is there?"
```

- Not queries, using the `~` operator, yield lines that don't match the query:

```
Executing Query for: ~(Alice)
~(Alice) occurs 9 times
(line 2) Her Daddy says when the wind blows
(line 3) through her hair, it looks almost alive,
(line 4) like a fiery bird in flight.
. . .
```

- Or queries, using the `|` operator, return lines matching either of two queries:

```
Executing Query for: (hair | Alice)
(hair | Alice) occurs 2 times
(line 1) Alice Emma has long flowing red hair.
(line 3) through her hair, it looks almost alive,
```

- And queries, using the `&` operator, return lines matching both queries:

```
Executing query for: (hair & Alice)
(hair & Alice) occurs 1 time
(line 1) Alice Emma has long flowing red hair.
```

Moreover, we want to be able to combine these operations, as in

```
fiery & bird | wind
```

We'll use normal C++ precedence rules (§ 4.1.2, p. 136) to evaluate compound expressions such as this example. Thus, this query will match a line in which both `fiery` and `bird` appear or one in which `wind` appears:

```

Executing Query for: ((fiery & bird) | wind)
((fiery & bird) | wind) occurs 3 times
(line 2) Her Daddy says when the wind blows
(line 4) like a fiery bird in flight.
(line 5) A beautiful fiery bird, he tells her,

```

Our output will print the query, using parentheses to indicate the way in which the query was interpreted. As with our original implementation, our system will display lines in ascending order and will not display the same line more than once.

15.9.1 An Object-Oriented Solution

We might think that we should use the `TextQuery` class from § 12.3.2 (p. 487) to represent our word query and derive our other queries from that class.

However, this design would be flawed. To see why, consider a `Not` query. A `Word` query looks for a particular word. In order for a `Not` query to be a kind of `Word` query, we would have to be able to identify the word for which the `Not` query was searching. In general, there is no such word. Instead, a `Not` query has a query (a `Word` query or any other kind of query) whose value it negates. Similarly, an `And` query and an `Or` query have two queries whose results it combines.

This observation suggests that we model our different kinds of queries as independent classes that share a common base class:

```

WordQuery // Daddy
NotQuery  // ~Alice
OrQuery   // hair | Alice
AndQuery  // hair & Alice

```

These classes will have only two operations:

- `eval`, which takes a `TextQuery` object and returns a `QueryResult`. The `eval` function will use the given `TextQuery` object to find the query's the matching lines.
- `rep`, which returns the string representation of the underlying query. This function will be used by `eval` to create a `QueryResult` representing the match and by the output operator to print the query expressions.

Abstract Base Class

As we've seen, our four query types are not related to one another by inheritance; they are conceptually siblings. Each class shares the same interface, which suggests that we'll need to define an abstract base class (§ 15.4, p. 610) to represent that interface. We'll name our abstract base class `Query_base`, indicating that its role is to serve as the root of our query hierarchy.

Our `Query_base` class will define `eval` and `rep` as pure virtual functions (§ 15.4, p. 610). Each of our classes that represents a particular kind of query must override these functions. We'll derive `WordQuery` and `NotQuery` directly from

KEY CONCEPT: INHERITANCE VERSUS COMPOSITION

The design of inheritance hierarchies is a complicated topic in its own right and well beyond the scope of this language Primer. However, there is one important design guide that is so fundamental that every programmer should be familiar with it.

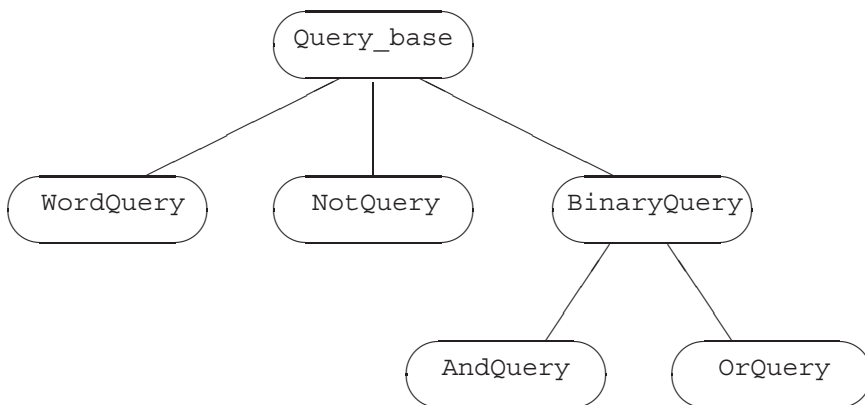
When we define a class as publicly inherited from another, the derived class should reflect an “Is A” relationship to the base class. In well-designed class hierarchies, objects of a publicly derived class can be used wherever an object of the base class is expected.

Another common relationship among types is a “Has A” relationship. Types related by a “Has A” relationship imply membership.

In our bookstore example, our base class represents the concept of a quote for a book sold at a stipulated price. Our `Bulk_quote` “is a” kind of quote, but one with a different pricing strategy. Our bookstore classes “have a” price and an ISBN.

`Query_base`. The `AndQuery` and `OrQuery` classes share one property that the other classes in our system do not: Each has two operands. To model this property, we’ll define another abstract base class, named `BinaryQuery`, to represent queries with two operands. The `AndQuery` and `OrQuery` classes will inherit from `BinaryQuery`, which in turn will inherit from `Query_base`. These decisions give us the class design represented in Figure 15.2.

Figure 15.2: `Query_base` Inheritance Hierarchy



Hiding a Hierarchy in an Interface Class

Our program will deal with evaluating queries, not with building them. However, we need to be able to create queries in order to run our program. The simplest way to do so is to write C++ expressions to create the queries. For example, we’d like to generate the compound query previously described by writing code such as

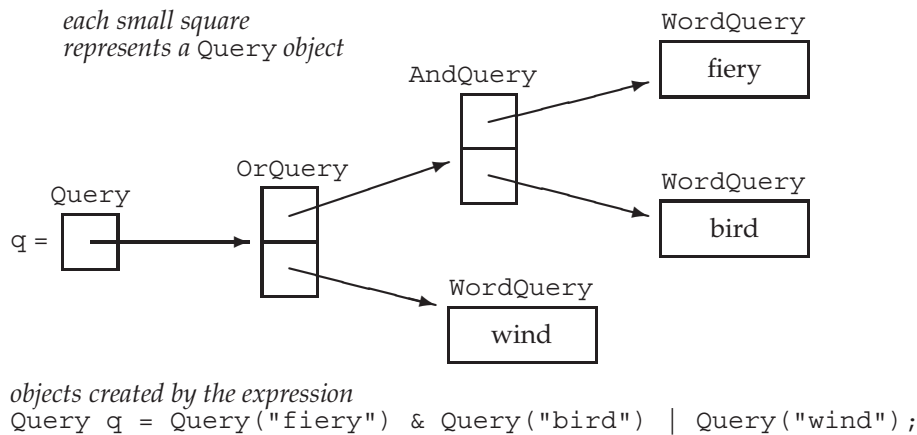
```
Query q = Query("fiery") & Query("bird") | Query("wind");
```

This problem description implicitly suggests that user-level code won't use the inherited classes directly. Instead, we'll define an interface class named `Query`, which will hide the hierarchy. The `Query` class will store a pointer to `Query_base`. That pointer will be bound to an object of a type derived from `Query_base`. The `Query` class will provide the same operations as the `Query_base` classes: `eval` to evaluate the associated query, and `rep` to generate a string version of the query. It will also define an overloaded output operator to display the associated query.

Users will create and manipulate `Query_base` objects only indirectly through operations on `Query` objects. We'll define three overloaded operators on `Query` objects, along with a `Query` constructor that takes a string. Each of these functions will dynamically allocate a new object of a type derived from `Query_base`:

- The `&` operator will generate a `Query` bound to a new `AndQuery`.
- The `|` operator will generate a `Query` bound to a new `OrQuery`.
- The `~` operator will generate a `Query` bound to a new `NotQuery`.
- The `Query` constructor that takes a string will generate a new `WordQuery`.

Figure 15.3: Objects Created by Query Expressions



Understanding How These Classes Work

It is important to realize that much of the work in this application consists of building objects to represent the user's query. For example, an expression such as the one above generates the collection of interrelated objects illustrated in Figure 15.3.

Once the tree of objects is built up, evaluating (or generating the representation of) a query is basically a process (managed for us by the compiler) of following these links, asking each object to evaluate (or display) itself. For example, if we

call `eval` on `q` (i.e., on the root of the tree), that call asks the `OrQuery` to which `q` points to `eval` itself. Evaluating this `OrQuery` calls `eval` on its two operands—on the `AndQuery` and the `WordQuery` that looks for the word `wind`. Evaluating the `AndQuery` evaluates its two `WordQuery`s, generating the results for the words `fiery` and `bird`, respectively.

When new to object-oriented programming, it is often the case that the hardest part in understanding a program is understanding the design. Once you are thoroughly comfortable with the design, the implementation flows naturally. As an aid to understanding this design, we've summarized the classes used in this example in Table 15.1 (overleaf).

EXERCISES SECTION 15.9.1

Exercise 15.31: Given that `s1`, `s2`, `s3`, and `s4` are all strings, determine what objects are created in the following expressions:

- (a) `Query(s1) | Query(s2) & ~ Query(s3);`
- (b) `Query(s1) | (Query(s2) & ~ Query(s3));`
- (c) `(Query(s1) & (Query(s2))) | (Query(s3) & Query(s4));`

15.9.2 The `Query_base` and `Query` Classes

We'll start our implementation by defining the `Query_base` class:

```
// abstract class acts as a base class for concrete query types; all members are private
class Query_base {
    friend class Query;
protected:
    using line_no = TextQuery::line_no; // used in the eval functions
    virtual ~Query_base() = default;
private:
    // eval returns the QueryResult that matches this Query
    virtual QueryResult eval(const TextQuery&) const = 0;
    // rep is a string representation of the query
    virtual std::string rep() const = 0;
};
```

Both `eval` and `rep` are pure virtual functions, which makes `Query_base` an abstract base class (§ 15.4, p. 610). Because we don't intend users, or the derived classes, to use `Query_base` directly, `Query_base` has no public members. All use of `Query_base` will be through `Query` objects. We grant friendship to the `Query` class, because members of `Query` will call the virtuals in `Query_base`.

The protected member, `line_no`, will be used inside the `eval` functions. Similarly, the destructor is protected because it is used (implicitly) by the destructors in the derived classes.

Table 15.1: Recap: Query Program Design

Query Program Interface Classes and Operations	
TextQuery	Class that reads a given file and builds a lookup map. This class has a query operation that takes a <code>string</code> argument and returns a <code>QueryResult</code> representing the lines on which that <code>string</code> appears (§ 12.3.2, p. 487).
QueryResult	Class that holds the results of a query operation (§ 12.3.2, p. 489).
Query	Interface class that points to an object of a type derived from <code>Query_base</code> .
Query q(s)	Binds the <code>Query</code> <code>q</code> to a new <code>WordQuery</code> holding the <code>string</code> <code>s</code> .
q1 & q2	Returns a <code>Query</code> bound to a new <code>AndQuery</code> object holding <code>q1</code> and <code>q2</code> .
q1 q2	Returns a <code>Query</code> bound to a new <code>OrQuery</code> object holding <code>q1</code> and <code>q2</code> .
~q	Returns a <code>Query</code> bound to a new <code>NotQuery</code> object holding <code>q</code> .
Query Program Implementation Classes	
Query_base	Abstract base class for the query classes.
WordQuery	Class derived from <code>Query_base</code> that looks for a given word.
NotQuery	Class derived from <code>Query_base</code> that represents the set of lines in which its <code>Query</code> operand does not appear.
BinaryQuery	Abstract base class derived from <code>Query_base</code> that represents queries with two <code>Query</code> operands.
OrQuery	Class derived from <code>BinaryQuery</code> that returns the union of the line numbers in which its two operands appear.
AndQuery	Class derived from <code>BinaryQuery</code> that returns the intersection of the line numbers in which its two operands appear.

The Query Class

The `Query` class provides the interface to (and hides) the `Query_base` inheritance hierarchy. Each `Query` object will hold a `shared_ptr` to a corresponding `Query_base` object. Because `Query` is the only interface to the `Query_base` classes, `Query` must define its own versions of `eval` and `rep`.

The `Query` constructor that takes a `string` will create a new `WordQuery` and bind its `shared_ptr` member to that newly created object. The `&`, `|`, and `~` operators will create `AndQuery`, `OrQuery`, and `NotQuery` objects, respectively. These operators will return a `Query` object bound to its newly generated object. To support these operators, `Query` needs a constructor that takes a `shared_ptr` to a `Query_base` and stores its given pointer. We'll make this constructor `private` because we don't intend general user code to define `Query_base` objects. Because this constructor is `private`, we'll need to make the operators `friends`.

Given the preceding design, the `Query` class itself is simple:

```
// interface class to manage the Query_base inheritance hierarchy
class Query {
    // these operators need access to the shared_ptr constructor
    friend Query operator~(const Query &);
    friend Query operator|(const Query&, const Query&);
    friend Query operator&(const Query&, const Query&);
```

```

public:
    Query(const std::string&); // builds a new WordQuery
    // interface functions: call the corresponding Query_base operations
    QueryResult eval(const TextQuery &t) const
        { return q->eval(t); }
    std::string rep() const { return q->rep(); }
private:
    Query(std::shared_ptr<Query_base> query): q(query) { }
    std::shared_ptr<Query_base> q;
};

```

We start by naming as friends the operators that create `Query` objects. These operators need to be friends in order to use the private constructor.

In the public interface for `Query`, we declare, but cannot yet define, the constructor that takes a `string`. That constructor creates a `WordQuery` object, so we cannot define this constructor until we have defined the `WordQuery` class.

The other two public members represent the interface for `Query_base`. In each case, the `Query` operation uses its `Query_base` pointer to call the respective (virtual) `Query_base` operation. The actual version that is called is determined at run time and will depend on the type of the object to which `q` points.

The Query Output Operator



The output operator is a good example of how our overall query system works:

```

std::ostream &
operator<<(std::ostream &os, const Query &query)
{
    // Query::rep makes a virtual call through its Query_base pointer to rep()
    return os << query.rep();
}

```

When we print a `Query`, the output operator calls the (public) `rep` member of class `Query`. That function makes a virtual call through its pointer member to the `rep` member of the object to which this `Query` points. That is, when we write

```

Query andq = Query(sought1) & Query(sought2);
cout << andq << endl;

```

the output operator calls `Query::rep` on `andq`. `Query::rep` in turn makes a virtual call through its `Query_base` pointer to the `Query_base` version of `rep`. Because `andq` points to an `AndQuery` object, that call will run `AndQuery::rep`.

EXERCISES SECTION 15.9.2

Exercise 15.32: What happens when an object of type `Query` is copied, moved, assigned, and destroyed?

Exercise 15.33: What about objects of type `Query_base`?

15.9.3 The Derived Classes

The most interesting part of the classes derived from `Query_base` is how they are represented. The `WordQuery` class is most straightforward. Its job is to hold the search word.

The other classes operate on one or two operands. A `NotQuery` has a single operand, and `AndQuery` and `OrQuery` have two operands. In each of these classes, the operand(s) can be an object of any of the concrete classes derived from `Query_base`: A `NotQuery` can be applied to a `WordQuery`, an `AndQuery`, an `OrQuery`, or another `NotQuery`. To allow this flexibility, the operands must be stored as pointers to `Query_base`. That way we can bind the pointer to whichever concrete class we need.

However, rather than storing a `Query_base` pointer, our classes will themselves use a `Query` object. Just as user code is simplified by using the interface class, we can simplify our own class code by using the same class.

Now that we know the design for these classes, we can implement them.

The `WordQuery` Class

A `WordQuery` looks for a given string. It is the only operation that actually performs a query on the given `TextQuery` object:

```
class WordQuery: public Query_base {
    friend class Query; // Query uses the WordQuery constructor
    WordQuery(const std::string &s): query_word(s) { }
    // concrete class: WordQuery defines all inherited pure virtual functions
    QueryResult eval(const TextQuery &t) const
        { return t.query(query_word); }
    std::string rep() const { return query_word; }
    std::string query_word; // word for which to search
};
```

Like `Query_base`, `WordQuery` has no public members; `WordQuery` must make `Query` a friend in order to allow `Query` to access the `WordQuery` constructor.

Each of the concrete query classes must define the inherited pure virtual functions, `eval` and `rep`. We defined both operations inside the `WordQuery` class body: `eval` calls the `query` member of its given `TextQuery` parameter, which does the actual search in the file; `rep` returns the string that this `WordQuery` represents (i.e., `query_word`).

Having defined the `WordQuery` class, we can now define the `Query` constructor that takes a string:

```
inline
Query::Query(const std::string &s): q(new WordQuery(s)) { }
```

This constructor allocates a `WordQuery` and initializes its pointer member to point to that newly allocated object.

The `NotQuery` Class and the `~` Operator

The `~` operator generates a `NotQuery`, which holds a `Query`, which it negates:

```

class NotQuery: public Query_base {
    friend Query operator~(const Query &);
    NotQuery(const Query &q): query(q) { }
    // concrete class: NotQuery defines all inherited pure virtual functions
    std::string rep() const {return "~(" + query.rep() + ")"; }
    QueryResult eval(const TextQuery&) const;
    Query query;
};

inline Query operator~(const Query &operand)
{
    return std::shared_ptr<Query_base>(new NotQuery(operand));
}

```

Because the members of `NotQuery` are all private, we start by making the `~` operator a friend. To rep a `NotQuery`, we concatenate the `~` symbol to the representation of the underlying `Query`. We parenthesize the output to ensure that precedence is clear to the reader.

It is worth noting that the call to `rep` in `NotQuery`'s own `rep` member ultimately makes a virtual call to `rep`: `query.rep()` is a nonvirtual call to the `rep` member of the `Query` class. `Query::rep` in turn calls `q->rep()`, which is a virtual call through its `Query_base` pointer.

The `~` operator dynamically allocates a new `NotQuery` object. The return (implicitly) uses the `Query` constructor that takes a `shared_ptr<Query_base>`. That is, the return statement is equivalent to

```

// allocate a new NotQuery object
// bind the resulting NotQuery pointer to a shared_ptr<Query_base>
shared_ptr<Query_base> tmp(new NotQuery(expr));
return Query(tmp); // use the Query constructor that takes a shared_ptr

```

The `eval` member is complicated enough that we will implement it outside the class body. We'll define the `eval` functions in § 15.9.4 (p. 647).

The BinaryQuery Class

The `BinaryQuery` class is an abstract base class that holds the data needed by the query types that operate on two operands:

```

class BinaryQuery: public Query_base {
protected:
    BinaryQuery(const Query &l, const Query &r, std::string s):
        lhs(l), rhs(r), opSym(s) { }
    // abstract class: BinaryQuery doesn't define eval
    std::string rep() const { return "(" + lhs.rep() + " "
                                + opSym + " "
                                + rhs.rep() + ")"; }

    Query lhs, rhs; // right- and left-hand operands
    std::string opSym; // name of the operator
};

```

The data in a `BinaryQuery` are the two `Query` operands and the corresponding operator symbol. The constructor takes the two operands and the operator symbol, each of which it stores in the corresponding data members.

To rep a `BinaryOperator`, we generate the parenthesized expression consisting of the representation of the left-hand operand, followed by the operator, followed by the representation of the right-hand operand. As when we displayed a `NotQuery`, the calls to `rep` ultimately make virtual calls to the `rep` function of the `Query_base` objects to which `lhs` and `rhs` point.



The `BinaryQuery` class does not define the `eval` function and so inherits a pure virtual. Thus, `BinaryQuery` is also an abstract base class, and we cannot create objects of `BinaryQuery` type.

The `AndQuery` and `OrQuery` Classes and Associated Operators

The `AndQuery` and `OrQuery` classes, and their corresponding operators, are quite similar to one another:

```
class AndQuery: public BinaryQuery {
    friend Query operator&(const Query&, const Query&);
    AndQuery(const Query &left, const Query &right):
        BinaryQuery(left, right, "&") { }

    // concrete class: AndQuery inherits rep and defines the remaining pure virtual
    QueryResult eval(const TextQuery&) const;
};

inline Query operator&(const Query &lhs, const Query &rhs)
{
    return std::shared_ptr<Query_base>(new AndQuery(lhs, rhs));
}

class OrQuery: public BinaryQuery {
    friend Query operator|(const Query&, const Query&);
    OrQuery(const Query &left, const Query &right):
        BinaryQuery(left, right, "|") { }

    QueryResult eval(const TextQuery&) const;
};

inline Query operator|(const Query &lhs, const Query &rhs)
{
    return std::shared_ptr<Query_base>(new OrQuery(lhs, rhs));
}
```

These classes make the respective operator a friend and define a constructor to create their `BinaryQuery` base part with the appropriate operator. They inherit the `BinaryQuery` definition of `rep`, but each overrides the `eval` function.

Like the `~` operator, the `&` and `|` operators return a `shared_ptr` bound to a newly allocated object of the corresponding type. That `shared_ptr` gets converted to `Query` as part of the return statement in each of these operators.

EXERCISES SECTION 15.9.3

Exercise 15.34: For the expression built in Figure 15.3 (p. 638):

- (a) List the constructors executed in processing that expression.
- (b) List the calls to `rep` that are made from `cout << q`.
- (c) List the calls to `eval` made from `q.eval()`.

Exercise 15.35: Implement the `Query` and `Query_base` classes, including a definition of `rep` but omitting the definition of `eval`.

Exercise 15.36: Put `print` statements in the constructors and `rep` members and run your code to check your answers to (a) and (b) from the first exercise.

Exercise 15.37: What changes would your classes need if the derived classes had members of type `shared_ptr<Query_base>` rather than of type `Query`?

Exercise 15.38: Are the following declarations legal? If not, why not? If so, explain what the declarations mean.

```
BinaryQuery a = Query("fiery") & Query("bird");
AndQuery b = Query("fiery") & Query("bird");
OrQuery c = Query("fiery") & Query("bird");
```

15.9.4 The `eval` Functions

The `eval` functions are the heart of our query system. Each of these functions calls `eval` on its operand(s) and then applies its own logic: The `OrQuery` `eval` operation returns the union of the results of its two operands; `AndQuery` returns the intersection. The `NotQuery` is more complicated: It must return the line numbers that are not in its operand's set.

To support the processing in the `eval` functions, we need to use the version of `QueryResult` that defines the members we added in the exercises to § 12.3.2 (p. 490). We'll assume that `QueryResult` has `begin` and `end` members that will let us iterate through the set of line numbers that the `QueryResult` holds. We'll also assume that `QueryResult` has a member named `get_file` that returns a `shared_ptr` to the underlying file on which the query was executed.



WARNING

Our `Query` classes use members defined for `QueryResult` in the exercises to § 12.3.2 (p. 490).

`OrQuery::eval`

An `OrQuery` represents the union of the results for its two operands, which we obtain by calling `eval` on each of its operands. Because these operands are `Query` objects, calling `eval` is a call to `Query::eval`, which in turn makes a virtual call to `eval` on the underlying `Query_base` object. Each of these calls yields a `QueryResult` representing the line numbers in which its operand appears. We'll combine those line numbers into a new set:

```

// returns the union of its operands' result sets
QueryResult
OrQuery::eval(const TextQuery& text) const
{
    // virtual calls through the Query members, lhs and rhs
    // the calls to eval return the QueryResult for each operand
    auto right = rhs.eval(text), left = lhs.eval(text);
    // copy the line numbers from the left-hand operand into the result set
    auto ret_lines =
        make_shared<set<line_no>>(left.begin(), left.end());
    // insert lines from the right-hand operand
    ret_lines->insert(right.begin(), right.end());
    // return the new QueryResult representing the union of lhs and rhs
    return QueryResult(rep(), ret_lines, left.get_file());
}

```

We initialize `ret_lines` using the `set` constructor that takes a pair of iterators. The `begin` and `end` members of a `QueryResult` return iterators into that object's set of line numbers. So, `ret_lines` is created by copying the elements from `left`'s set. We next call `insert` on `ret_lines` to insert the elements from `right`. After this call, `ret_lines` contains the line numbers that appear in either `left` or `right`.

The `eval` function ends by building and returning a `QueryResult` representing the combined match. The `QueryResult` constructor (§ 12.3.2, p. 489) takes three arguments: a `string` representing the query, a `shared_ptr` to the set of matching line numbers, and a `shared_ptr` to the vector that represents the input file. We call `rep` to generate the `string` and `get_file` to obtain the `shared_ptr` to the file. Because both `left` and `right` refer to the same file, it doesn't matter which of these we use for `get_file`.

AndQuery::eval

The `AndQuery` version of `eval` is similar to the `OrQuery` version, except that it calls a library algorithm to find the lines in common to both queries:

```

// returns the intersection of its operands' result sets
QueryResult
AndQuery::eval(const TextQuery& text) const
{
    // virtual calls through the Query operands to get result sets for the operands
    auto left = lhs.eval(text), right = rhs.eval(text);
    // set to hold the intersection of left and right
    auto ret_lines = make_shared<set<line_no>>();
    // writes the intersection of two ranges to a destination iterator
    // destination iterator in this call adds elements to ret
    set_intersection(left.begin(), left.end(),
                    right.begin(), right.end(),
                    inserter(*ret_lines, ret_lines->begin()));
    return QueryResult(rep(), ret_lines, left.get_file());
}

```


Here we use the library `set_intersection` algorithm, which is described in Appendix A.2.8 (p. 880), to merge these two sets.

The `set_intersection` algorithm takes five iterators. It uses the first four to denote two input sequences (§ 10.5.2, p. 413). Its last argument denotes a destination. The algorithm writes the elements that appear in both input sequences into the destination.

In this call we pass an insert iterator (§ 10.4.1, p. 401) as the destination. When `set_intersection` writes to this iterator, the effect will be to insert a new element into `ret_lines`.

Like the `OrQuery eval` function, this one ends by building and returning a `QueryResult` representing the combined match.

NotQuery::eval

`NotQuery` finds each line of the text within which the operand is not found:

```
// returns the lines not in its operand's result set
QueryResult
NotQuery::eval(const TextQuery& text) const
{
    // virtual call to eval through the Query operand
    auto result = query.eval(text);
    // start out with an empty result set
    auto ret_lines = make_shared<set<line_no>>();
    // we have to iterate through the lines on which our operand appears
    auto beg = result.begin(), end = result.end();
    // for each line in the input file, if that line is not in result,
    // add that line number to ret_lines
    auto sz = result.get_file()->size();
    for (size_t n = 0; n != sz; ++n) {
        // if we haven't processed all the lines in result
        // check whether this line is present
        if (beg == end || *beg != n)
            ret_lines->insert(n); // if not in result, add this line
        else if (beg != end)
            ++beg; // otherwise get the next line number in result if there is one
    }
    return QueryResult(rep(), ret_lines, result.get_file());
}
```

As in the other `eval` functions, we start by calling `eval` on this object's operand. That call returns the `QueryResult` containing the line numbers on which the operand appears, but we want the line numbers on which the operand does not appear. That is, we want every line in the file that is not already in `result`.

We generate that `set` by iterating through sequential integers up to the size of the input file. We'll put each number that is not in `result` into `ret_lines`. We position `beg` and `end` to denote the first and one past the last elements in `result`. That object is a `set`, so when we iterate through it, we'll obtain the line numbers in ascending order.

The loop body checks whether the current number is in `result`. If not, we add that number to `ret_lines`. If the number is in `result`, we increment `beg`, which is our iterator into `result`.

Once we've processed all the line numbers, we return a `QueryResult` containing `ret_lines`, along with the results of running `rep` and `get_file` as in the previous `eval` functions.

EXERCISES SECTION 15.9.4

Exercise 15.39: Implement the `Query` and `Query_base` classes. Test your application by evaluating and printing a query such as the one in Figure 15.3 (p. 638).

Exercise 15.40: In the `OrQuery` `eval` function what would happen if its `rhs` member returned an empty set? What if its `lhs` member did so? What if both `rhs` and `lhs` returned empty sets?

Exercise 15.41: Reimplement your classes to use built-in pointers to `Query_base` rather than `shared_ptr`s. Remember that your classes will no longer be able to use the synthesized copy-control members.

Exercise 15.42: Design and implement one of the following enhancements:

- (a) Print words only once per sentence rather than once per line.
- (b) Introduce a history system in which the user can refer to a previous query by number, possibly adding to it or combining it with another.
- (c) Allow the user to limit the results so that only matches in a given range of lines are displayed.

CHAPTER SUMMARY

Inheritance lets us write new classes that share behavior with their base class(es) but override or add to that behavior as needed. Dynamic binding lets us ignore type differences by choosing, at run time, which version of a function to run based on an object's dynamic type. The combination of inheritance and dynamic binding lets us write type-independent, programs that have type-specific behavior.

In C++, dynamic binding applies *only* to functions declared as `virtual` and called through a reference or pointer.

A derived-class object contains a subobject corresponding to each of its base classes. Because every derived object contains a base part, we can convert a reference or pointer to a derived-class type to a reference or pointer to an accessible base class.

Inherited objects are constructed, copied, moved, and assigned by constructing, copying, moving, and assigning the base part(s) of the object before handling the derived part. Destructors execute in the opposite order; the derived type is destroyed first, followed by destructors for the base-class subobjects. Base classes usually should define a virtual destructor even if the class otherwise has no need for a destructor. The destructor must be virtual if a pointer to a base is ever deleted when it actually addresses a derived-class object.

A derived class specifies a protection level for each of its base class(es). Members of a `public` base are part of the interface of the derived class; members of a `private` base are inaccessible; members of a `protected` base are accessible to classes that derive from the derived class but not to users of the derived class.

DEFINED TERMS

abstract base class Class that has one or more pure virtual functions. We cannot create objects of an abstract base-class type.

accessible Base class member that can be used through a derived object. Accessibility depends on the access specifier used in derivation list of the derived class and the access level of the member in the base class. For example, a `public` member of a class that is inherited via `public` inheritance is accessible to users of the derived class. A `public` base class member is inaccessible if the inheritance is `private`.

base class Class from which other classes inherit. The members of the base class become members of the derived class.

class derivation list List of base classes, each of which may have an optional access level, from which a derived class inher-

its. If no access specifier is provided, the inheritance is `public` if the derived class is defined with the `struct` keyword, and is `private` if the class is defined with the `class` keyword.

derived class Class that inherits from another class. A derived class can override the virtuals of its base and can define new members. A derived-class scope is nested in the scope of its base class(es); members of the derived class can use members of the base class directly.

derived-to-base conversion Implicit conversion of a derived object to a reference to a base class, or of a pointer to a derived object to a pointer to the base type.

direct base class Base class from which a derived class inherits directly. Direct base classes are specified in the derivation list of

the derived class. A direct base class may itself be a derived class.

dynamic binding Delaying until run time the selection of which function to run. In C++, dynamic binding refers to the run-time choice of which virtual function to run based on the underlying type of the object to which a reference or pointer is bound.

dynamic type Type of an object at run time. The dynamic type of an object to which a reference refers or to which a pointer points may differ from the static type of the reference or pointer. A pointer or reference to a base-class type can refer to an object of derived type. In such cases the static type is reference (or pointer) to base, but the dynamic type is reference (or pointer) to derived.

indirect base class Base class that does not appear in the derivation list of a derived class. A class from which the direct base class inherits, directly or indirectly, is an indirect base class to the derived class.

inheritance Programming technique for defining a new class (known as a derived class) in terms of an existing class (known as the base class). The derived class inherits the members of the base class.

object-oriented programming Method of writing programs using data abstraction, inheritance, and dynamic binding.

override Virtual function defined in a derived class that has the same parameter list as a virtual in a base class overrides the base-class definition.

polymorphism As used in object-oriented programming, refers to the ability to obtain type-specific behavior based on the dynamic type of a reference or pointer.

private inheritance In private inheritance, the public and protected members of the base class are private members of the derived.

protected access specifier Members defined after the `protected` keyword may be accessed by the members and friends of a derived class. However, these members are only accessible through derived objects. protected members are not accessible to ordinary users of the class.

protected inheritance In protected inheritance, the protected and public members of the base class are protected members of the derived class.

public inheritance The public interface of the base class is part of the public interface of the derived class.

pure virtual Virtual function declared in the class header using `= 0` just before the semicolon. A pure virtual function need not be (but may be) defined. Classes with pure virtuals are abstract classes. If a derived class does not define its own version of an inherited pure virtual, then the derived class is abstract as well.

refactoring Redesigning programs to collect related parts into a single abstraction, replacing the original code with uses of the new abstraction. Typically, classes are refactored to move data or function members to the highest common point in the hierarchy to avoid code duplication.

run-time binding See dynamic binding.

sliced down What happens when an object of derived type is used to initialize or assign an object of the base type. The derived portion of the object is “sliced down,” leaving only the base portion, which is assigned to the base.

static type Type with which a variable is defined or that an expression yields. Static type is known at compile time.

virtual function Member function that defines type-specific behavior. Calls to a virtual made through a reference or pointer are resolved at run time, based on the type of the object to which the reference or pointer is bound.